Unit - 1

Common Terms

Internet

A *global* network of networks of interconnected computers.

Web

also referred to formally as World Wide Web (www) is a *collection of information* that is accessed via the Internet.

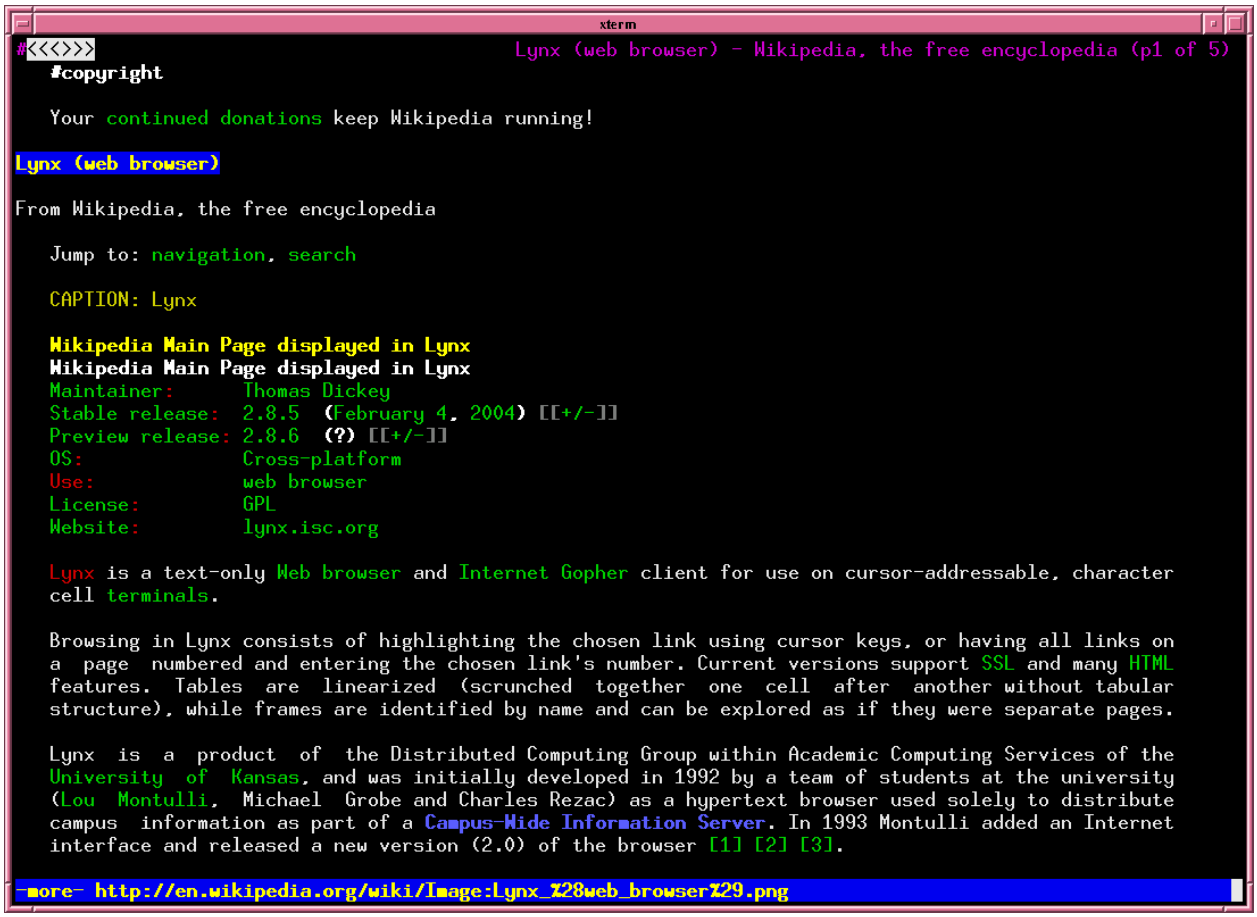
How does WWW work

Client Server Architecture

- Every time you go to a website, you become a *client*.
- Your computer sends a *request* to another computer on the internet. This computer is called a *server*
- Then the server receives the request, and sends an appropriate *response*
- How these requests and responses are formatted are defined by the *Hyper Text Markup Language* (HTTP)

Web Browsers

- You know what a web browser is. It's just your Google chrome, Brave, Vivaldi, Mozilla Firefox, Opera, etc.
- When web browsers were first invented, they were mostly text based.
- Almost all modern web browsers are much more sophisticated and look way better. They offer support for mouse, 3D rendering, etc.
- Lynx: an old text based browser:



URLs

- **U**niversal **R**esource **L**ocator
- Let's take a URL you see everyday and break it down.

```
https://en.wikipedia.org/wiki/PES_University%2C_EC_Campus
```

https://

- This defined the protocol. Sometimes, it's only *http*
- In this case, it's *https*. The *s* stands for *secure*. This means that the server between the client and the server, aka your computer and wikipedia's server is encrypted and nobody that is on the path of the connection between you and wikipedia can see the contents of the connection

en.wikipedia.org

- This defines the *domain name*
- More about domain names in a second.
- This domain name can be substituted by a raw IP address. Also more about that in a second

/wiki/PES_University%2C_EC_Campus

- This is the internal address of the resource you are trying to access *in the server*
- It defines what web page you are seeing

DNS

- **D**omain **N**ame **S**ervice
- Before DNS, we gotta know what an *IP address* is

IP Address

- The structure of the internet is kinda like homes in a neighbourhood
- Just like how every house has an address, every computer on the internet must have an *IP address*
- There are 2 versions of the IP address mainly in use, *ipv4* and *ipv6*
- *ipv4* is mostly used and it has 4 subparts. A typical *ip address* looks something like this: **1.6.180.187**
- You can see how the 4 parts are separated by a period (.)

Back to DNS

- Imagine you are trying to connect to wikipedia again.
- The wikipedia server, since it's just a computer on the internet, has an IP address
- If you want to go to a friend's home, you go to his address. Similarly, in order to go to a server, you need it's address
- And it is true that you can just use an IP address in a URL. This is sometimes used when buying a domain name is too expensive and you're too broke. Like in PES, in order to access the WiFi captive portal, you enter http://192.168.254.1:8090/httpclient.html into your web browser. The numbers *192.168.254.1* is the IP address of the server you're connecting to, which is basically some old computer located somewhere in PES.
- Note that this is a *local IP address*, this only works inside PES University when you're connected to PES networks, either through ethernet, or through WiFi.

Back to DNS again

- You can't practically remember all the IP addresses on the internet.
- You can't expect people to remember random shit, unless you're the Indian education system
- So we use this thing called DNS, where we give a nickname to a server's IP address. You just need to remember the nickname. This nickname looks something like *en.wikipedia.org*
- This nickname is called a **domain**
- Now when you enter a domain into your web browser, and hit enter, the browser first goes to a *domain resolver* instead of directly to the server.

- This *domain resolver* is also a server, aka a computer. The job of this domain resolver is to keep a track of IP addresses and their corresponding nicknames (domains)
- So your web browser(your computer) asks the domain resolver what the IP address is of the domain you entered, gets the IP address and then goes to the original server you intended to go to.
- When servers are involved, ofc there's business involved. There are various providers for DNS. They just provide the servers for the *domain resolver*. Examples are Google DNS, Cloudflare DNS, etc.
- Now how do you get to the DNS server? you can't have a nickname for it- that'd be a name loop.
- You instead just remember the IP for it, and the IPs of DNS servers are designed in such a way that they are easy to remember. For example, the IP address of Google DNS is 8.8.8.8 and Cloudflare DNS is 1.1.1.1

HTML

- Every webpage on the internet starts with HTML.
- HTML stands for *Hyper Text Markup Language* and defines the skeleton of your webpage.
- The looks of your website are defined by *CSS* or *cascading style sheets*
- The logic is defined by *javascript*

Syntaxes

- Every HTML document has tags. Every element is a *tag*
- Every *tag* has to be opened, and then closed.
- These tags can contain attributes. These are all structured in this way:

```HTML
<tag attributes_go_here>whatever is inside the tag </tag>
^ tag open                                          ^ tag close
```

- For example:

```HTML
<h1>This is a heading</h1>
```

- `<h1>` is a heading tag. Every tag must be closed by a closing tag. A closing tag is just the opening tag but with a `/` at the beginning of it's name.
- There are also self closing tags that do not need to be closed. The self closing tags have a `/` at the end of the tag itself. These require no closing tags:

```HTML
<img href="url_of_some_image"/>
```

Different HTML tags

Here's some HTML code. It's pretty descriptive. The tags are short forms of what they do:

```HTML
<!DOCTYPE html>
<html lang="en">

<head>
  <title>this is the title. This is the name that appears on the tab</title>
</head>

<!-- The head section contains some metadata about your web page. These include imports of CSS, JS and other files, Titles, favicon definition, and some data for SEO as well -->
<!-- SEO stands for search engine optimization. It is not relevant as of now but you can always google if you're curious -->
<!-- These are comments btw -->


<!-- The body tag defines the main body of your website. These tags are rendered to your web page -->

<body>
  <h1>Heading level 1</h1>
  <h2>Heading level 2</h2>
  <h3>Heading level 3</h3>
  <h4>Heading level 4</h4>
  <h5>Heading level 5</h5>
  <h6>Heading level 6</h6>



  <p>This is a paragraph. Continuous spaces like     this and
    line breaks are not conserved here</p>
  <p>If you want a line break, you can use this tag: <br />And this should appear on the next line</p>

  <pre>This is a preformatted text tag. continuous spaces
    and line breaks are conserved here</pre>

  <!-- unordered list -->
```

```
  <ul>
    <li>list item 1</li>
    <li>list item 2</li>
    <li>list item 3</li>
  </ul>


  <!-- ordered list -->
  <ol>
    <li>list item 1</li>
    <li>list item 2</li>
    <li>list item 3</li>
  </ol>


  <!-- description list -->
  <dl>
    <dt>Description title 1</dt>
    <dd>Data Definition 1</dd>
    <dt>Description title 2</dt>
    <dd>Data Definition 2</dd>
    <dt>Description title 2</dt>
    <dd>Data Definition 2</dd>
  </dl>

</body>

</html>
```

And this is how it renders as:

Heading level 1

Heading level 2

Heading level 3

Heading level 4

Heading level 5

Heading level 6

This is a paragraph. Continuous spaces like this and line breaks are not conserved here

If you want a line break, you can use this tag:
And this should appear on the next line

```
This is a preformatted text tag. continuous spaces
    and line breaks are conserved here
```

- list item 1
- list item 2
- list item 3

1. list item 1
2. list item 2
3. list item 3

Description title 1
: Data Definition 1
Description title 2
: Data Definition 2
Description title 2
: Data Definition 2

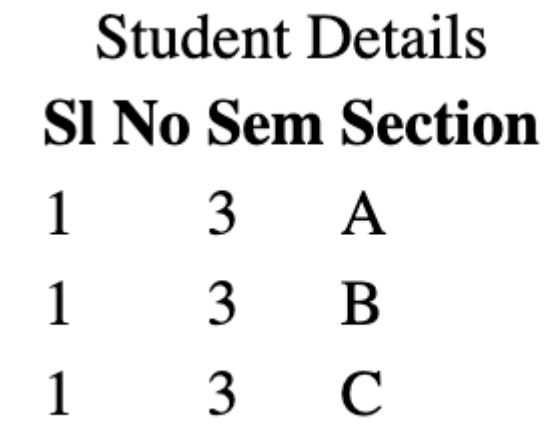
HTML Table

This is how you make tables in html

```HTML
<!DOCTYPE html>
<html lang="en">

<head>
  <title>Table demo</title>
</head>
```

```
<body>
  <table>
    <caption>Student Details</caption>

    <!-- table row -->
    <!-- first row -->
    <tr>
      <!-- table header -->
      <th>Sl No</th>
      <th>Sem</th>
      <th>Section</th>
    </tr>

    <!-- second row -->
    <tr>
      <!-- table data -->
      <td>1</td>
      <td>3</td>
      <td>A</td>
    </tr>

    <!--third row-->
    <tr>
      <!-- table data -->
      <td>1</td>
      <td>3</td>
      <td>B</td>
    </tr>


    <!-- fourth row -->
    <tr>
      <!-- table data -->
      <td>1</td>
      <td>3</td>
      <td>C</td>
    </tr>

  </table>
</body>

</html>
```

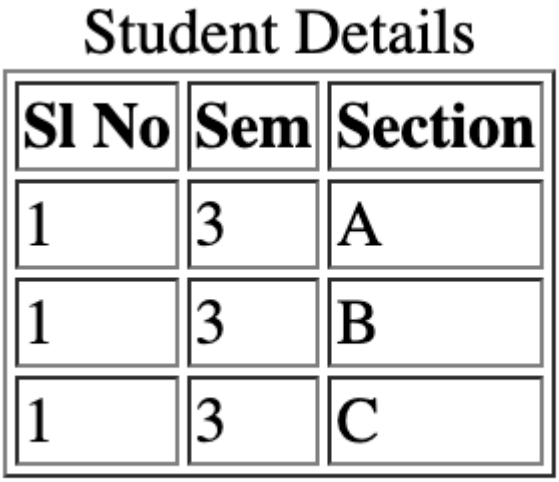
And it renders like this:



- It doesn't add a border by default. You can add the attribute: `border="1"` in the `<table>` tag to add the border. Like this:

```HTML
<table border="1">
 <!— stuff inside the table here —>
</table>
```

And it renders like this:



- Increasing the border number in `border="1"` increases the width of the border.

Info

You can also make particular columns/rows span multiple columns and rows by adding the attribute `rowspan="2"` or `colspan="3"` to your `td` or `th` elements (2

and 3 are just arbitrary numbers. They can be any number of rows or columns)

Subscript and Super Script

Here's the code:

```HTML
<!DOCTYPE html>
<html lang="en">

<head>
  <title>Sub and Super Script</title>
</head>

<body>
  <p>
    This is normal text
    <sub>subscript</sub>
    <sup>superscript</sup>
  </p>
</body>

</html>
```

Renders like:

This is normal text $_{\text{subscript}}$ $^{\text{superscript}}$

Info

HTML and *XML* are two different markup languages. The basic structure of both are the same. The main difference between these two is that the tags in HTML are pre defined by the HTML specification. In XML, you can define your own tags and it's mainly used to transmit data across the internet in a formatted way. Another way to transmit data is to use JSON formatting. We'd be doing this later in the course.

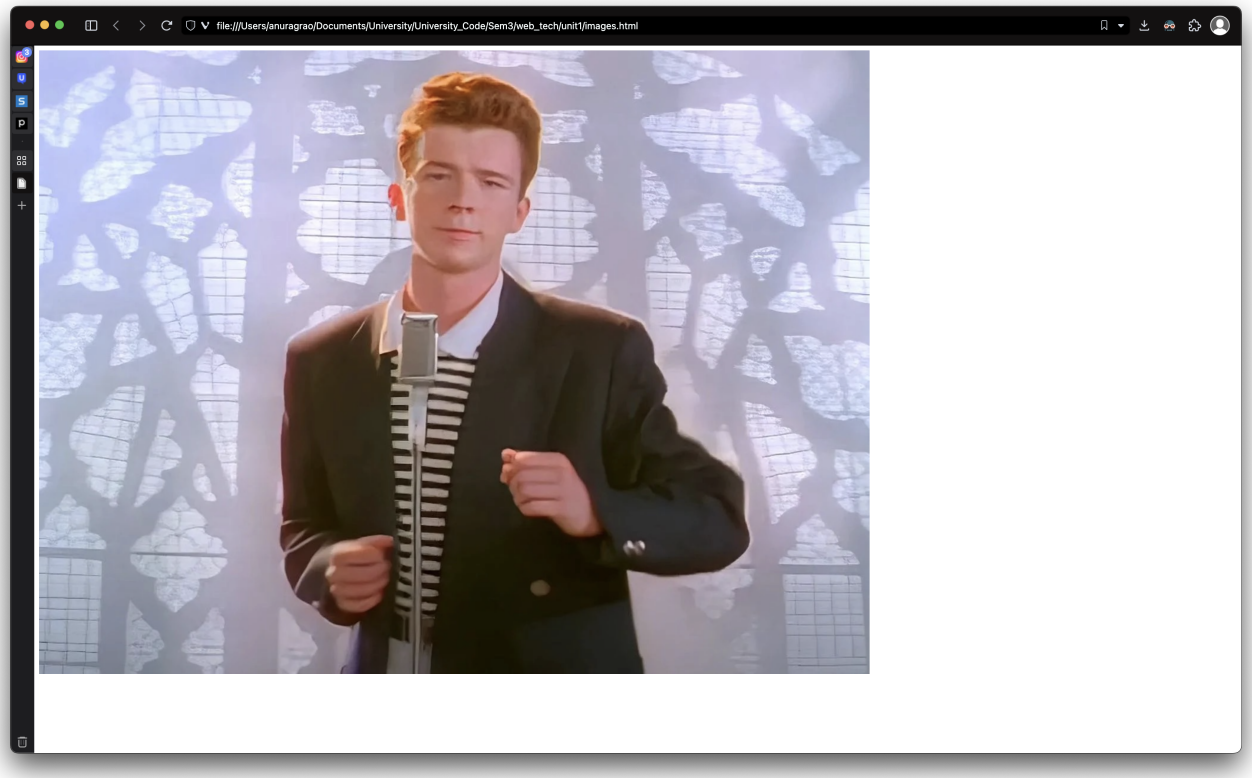
Images

- You can add images using the `img` tag. This `img` tag should have an `src` attribute.
- `src` stands for source and must contain the file address or a url to the image you want to add.
- The `alt` text is rendered if your image address can't be accessed for some reason.
- You can also specify `height` and `width` attributes in the image tag. This takes the values in pixels. If you only specify one of them, either `height` or `width`, then the image will be resized preserving the aspect ratio.

Code:

```HTML
<!DOCTYPE html>
<html lang="en">

<head>
  <title>Images demo</title>
</head>

<body>

  <img src="sample_image.jpeg"
    alt="this text will be rendered if for some reason, your src is wrong and cannot be accessed">
</body>

</html>
```

renders like this in the browser:



HTML Forms

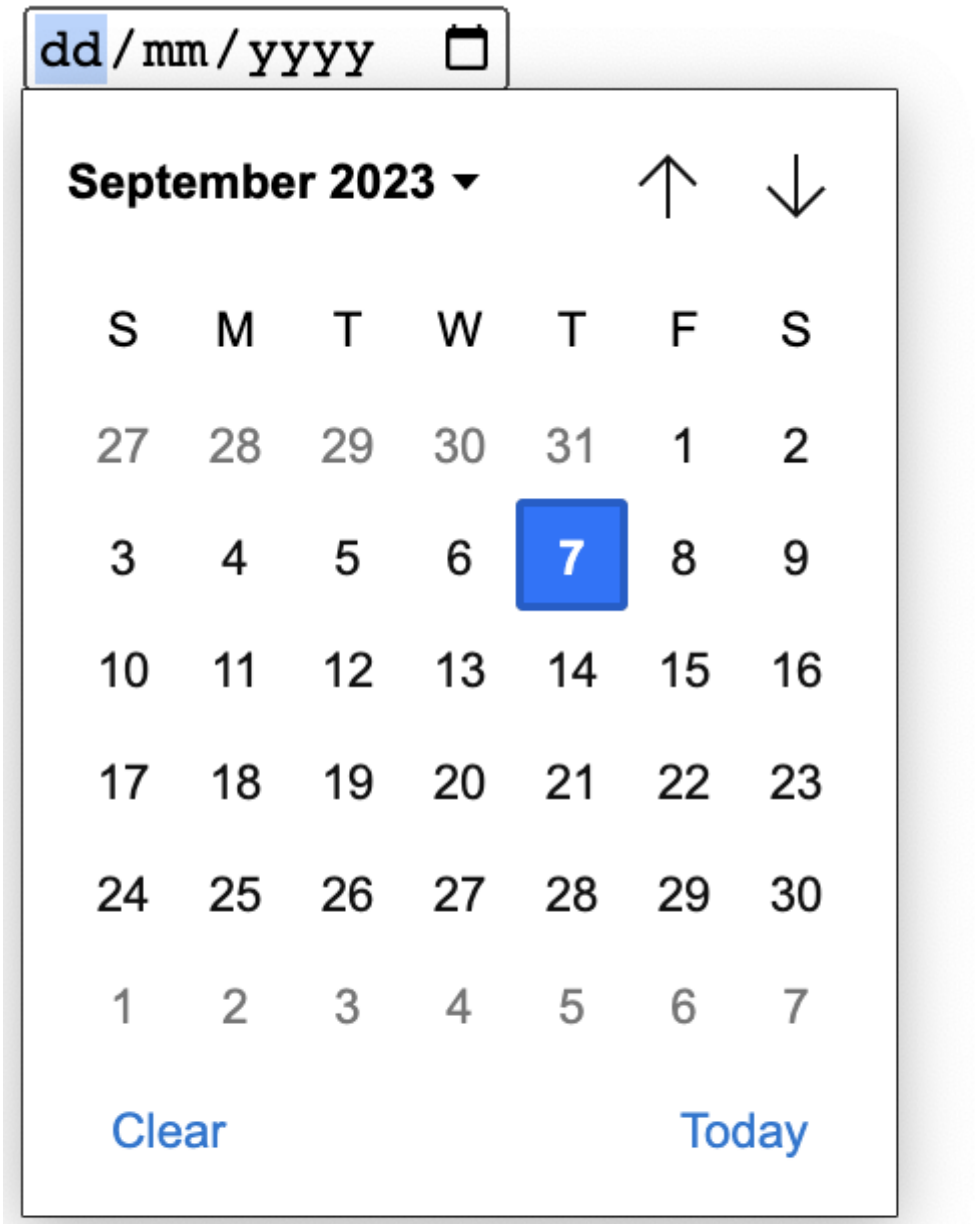
- HTML forms allow you to build ...🥁 ... a form
- It allows you to ask the user for multiple data fields, and then hit submit.
- At it's core, an HTML form is just a collection of a set of tags called *form controls* or *widgets*. These can be multi line text fields, drop down boxes, checkboxes etc. They are mostly created using the `<input>` tag.
- Here's an example of a super basic HTML form:

```HTML
<form>
  <label for="name">Name:</label>
  <input type="text" id="name" name="name">
  <input type="submit" value="Submit">
</form>
```

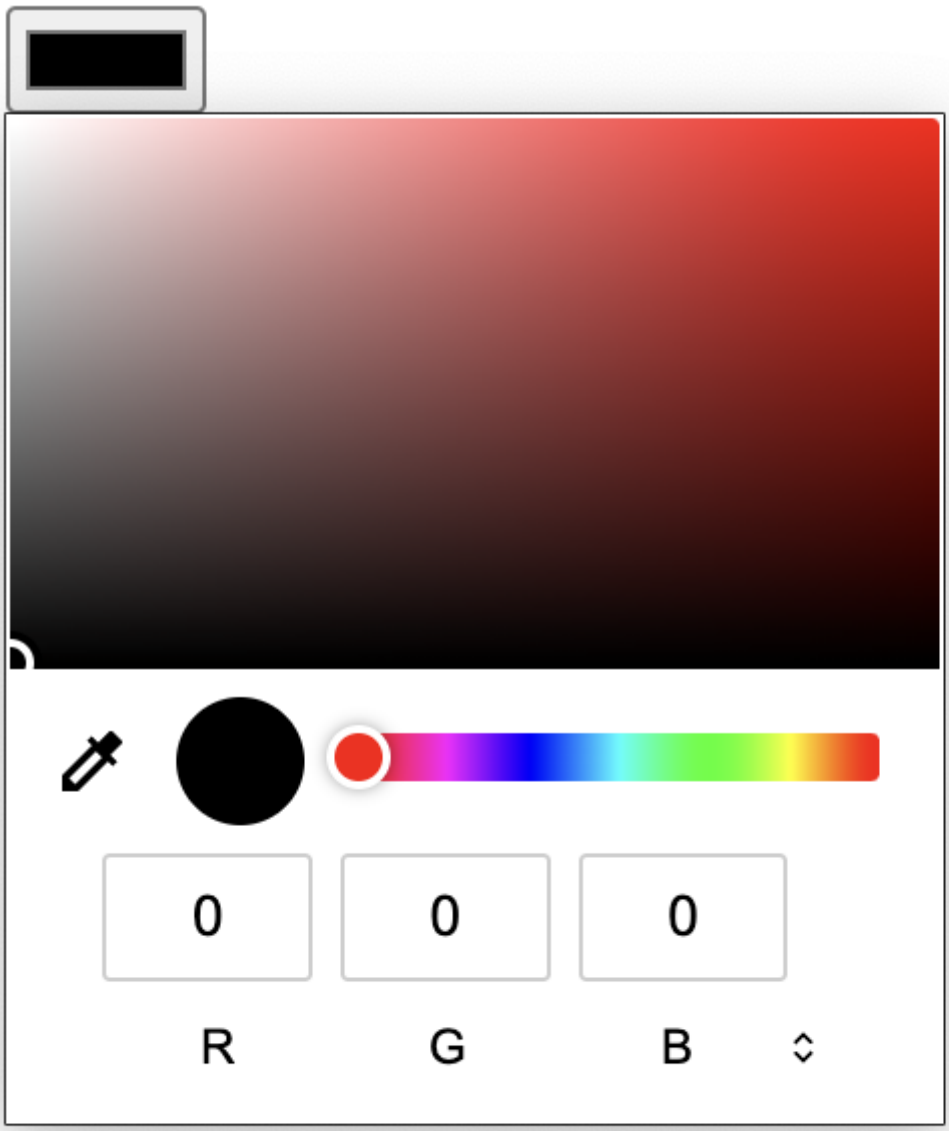
Name: Submit

- Here you have a text input field of the `type="text"`. This defines the type of the input. This can be text, number, data etc. HTML will make appropriate arrangements according to the input type.

- For example, if you give `type="date"`, then it'll make a date selection kind of thing to select a date:



- Similarly, `type="color"` gives something like this:



- You can also have an `type="email"` that validates the email in the input field.
- Radio buttons and check boxes can be implemented like this:

```HTML
<input type="radio" name="Subject" value="Web Tech" required>
<label for="Web Tech">Web Tech</label>
<input type="radio" name="Subject" value="DSA" required>
<label for="DSA">DSA</label>
<input type="radio" name="Subject" value="AFLL" required>
<label for="AFLL">AFLL</label>
<input type="radio" name="Subject" value="DDCO" required>
<label for="DDCO">DDCO</label>
<input type="radio" name="Subject" value="SDS" required>
<label for="SDS">SDS</label>
```

- The `name` attribute of mutually exclusive radio buttons must be the same.

- By 'mutually exclusive', I mean that only one of these radio buttons can be checked at a time.
- The `value` attribute is not rendered on the screen. This is the data sent to the server when you have that radio button checked. For example, if you have the `DSA` radio button checked, it would appear in the data as:

```
{
 ... other data here
 Subject: "DSA",
 ... other data here
}
```

- This data is in the JSON format. JSON is just a set of `key:value` pairs. It's kinda like a python dictionary.

There are many other input types that include:

1. `range` : gives you a slider for selection
2. `url` : for urls
3. `search` : for search boxes
4. `month` : for month selection
5. `time` : for time selection
6. `week` : for week selection
7. `datetime` : for date and time
8. `password` : does not show the letters of what you entered

Info

You can add a `placeholder` attribute to text inputs to have a greyed out placeholder text on your inputs. Something like 'input your phone number', etc.

There are a few other important ones that are mentioned in the slides. They are:

Text

```HTML
<input type="text" name="some_name">
```

creates a box for text input

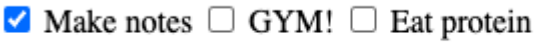
- *The default size is 20*
- This size can be changed with the `size` attribute

- If you type more characters than can fit in this box, then it'll scroll to the left.
- You can prevent this by setting a `maxlength` attribute. Something like this:

```HTML
<input type="number" name = "phone_number" maxlength="10"
```

Checkbox

```HTML
<input type="checkbox" name="todo" value="make_notes" checked>
<span>Make notes</span>
<input type="checkbox" name="todo" value="go_to_gym">
<span>GYM!</span>
<input type="checkbox" name="todo" value="eat_protein">
<span>Eat protein</span>
```



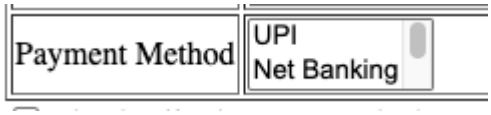
- Every checkbox requires a `value` attribute. This is the data sent to the server when the checkbox is checked
- By default, checkboxes aren't checked. You can make them checked using the `checked` attribute. You can also do `checked="checked"` but just `checked` will also do the job

Drop downs

```HTML
<select name="payment" required>
 <option value="UPI">UPI</option>
 <option value="Net Banking">Net Banking</option>
 <option value="Credit Card">Credit Card</option>
 <option value="Debit Card">Debit Card</option>
</select>
```

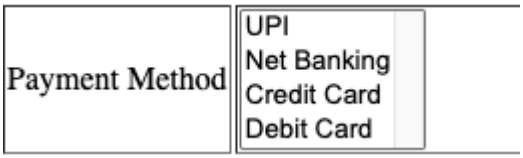
- Every `option` in the `select` tag will be an option on the drop down. The options are mutually exclusive, like radio buttons. Only one can be selected at a time.
- If you want multiple items to be selectable, you have to use the `multiple` attribute on the `select` tag.
- You can also increase the size of the drop down menu visible window by giving the `size="2"` attribute. This number is the number of options available in the visible input window. For example:

```HTML
<select name="payment" required size="2">
 <option value="UPI">UPI</option>
 <option value="Net Banking">Net Banking</option>
 <option value="Credit Card">Credit Card</option>
 <option value="Debit Card">Debit Card</option>
</select>
```



- If you use the `multiple` attribute, then all options will be visible at once.

```HTML
<select name="payment" required multiple>
 <option value="UPI">UPI</option>
 <option value="Net Banking">Net Banking</option>
 <option value="Credit Card">Credit Card</option>
 <option value="Debit Card">Debit Card</option>
</select>
```



- You can also preselect a default option by using the attribute `selected` in one of the options (or multiple if your drop down accepts multiple selections)

Text Area

```HTML
<textarea name = "some_text_area" rows="10" cols="100">some pre existing text</textarea>
```

- As the name suggests, this is kinda like a normal textbox but it's larger. You can specify the number of rows and columns (height and width) of the text area.
- If the text is overfilled, it scrolls downwards *unlike* a normal textbox that scrolls horizontally.

Range

```HTML
<input type="range" name="how_many_bun_samosa" min="1" max="69">
```

- This range is shown as a slider.
- You can specify the `min` and `max` attributes to specify the min and max value of your slider.

The `reset` and `submit` buttons

```HTML
<input type="reset" name="reset_form" value="Reset">
<input type="submit" name="submit_form" value="Submit">
```

- When the submit button is clicked, then it does 2 things:

1. Encode the data of the form into the appropriate format
2. Send a request to the server specified in the `action` attribute of the `form` tag

- The reset button resets all the input fields and gets them back to their default unfilled state.
- Every HTML form must contain a submit button. It will not cause a visible error in your website but what's the point of a form if you can't submit it.

What happens when you hit the submit button?

- The data that you filled in the form has to go somewhere.
- Usually, when you hit submit, this data is sent to a backend server - A separate server. The logic and what to do with this data is handled by the backend server.
- The transmission of data is ofc sent through HTTP(S)
- It can be sent via different `HTTP Methods`

HTTP Methods

- There are pre defined methods that HTTP talks in. They are `GET`, `POST` etc. These are the 2 most commonly used ones. There are many more like `PATCH`, `DELETE`,

etc.

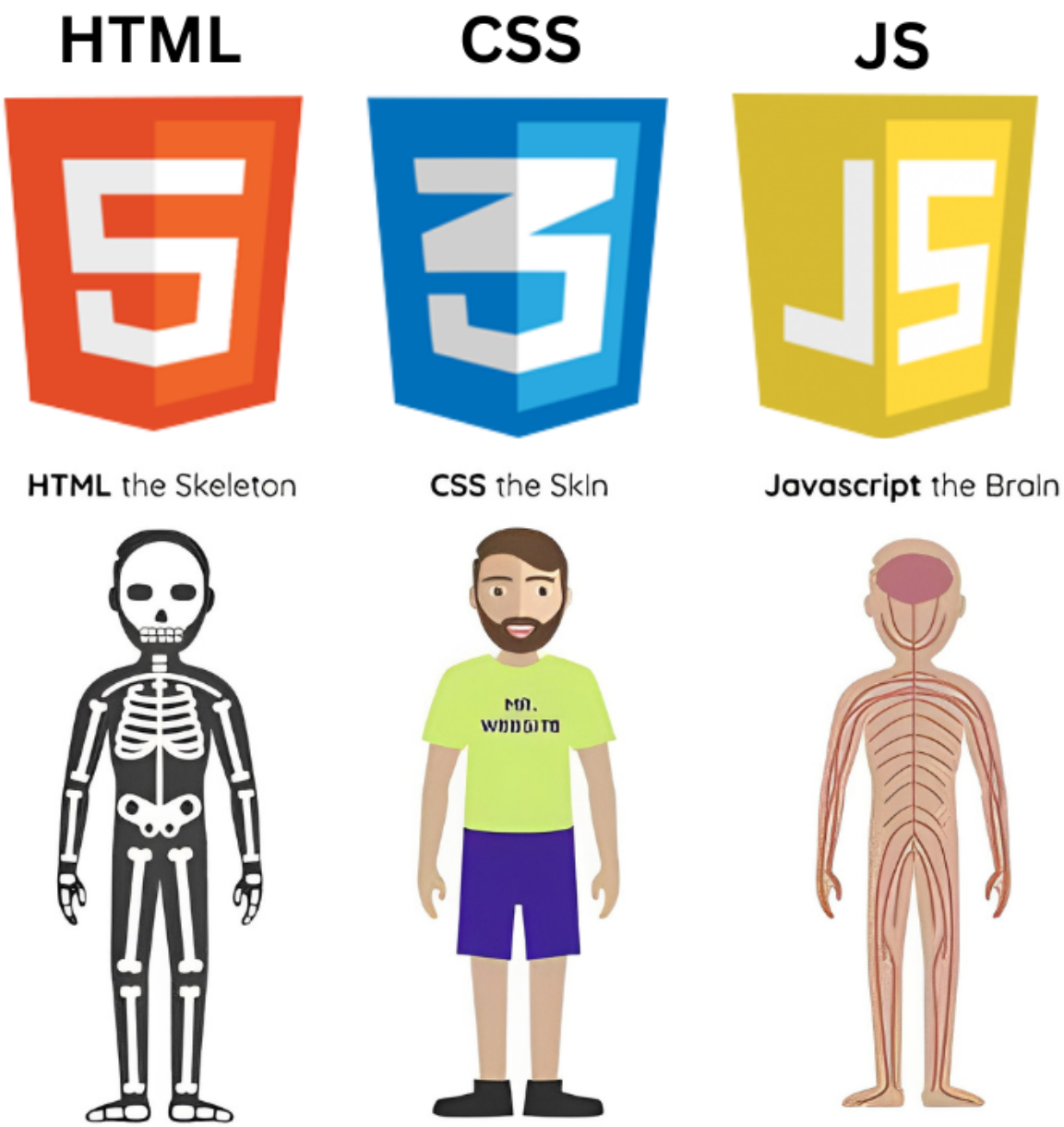
- They are also referred as *HTTP verbs*
- `GET` is used for getting data from the backend server. Whenever you go to a website, your browser by default sends a `GET` request to the web server. The web server then returns the website to the browser - which you see.
- The `POST` method is used to *post* data to the backend server. This is used when your frontend (website) wants to send data to a backend server, *like when you submit a form*
- You can also use the `GET` method but it appends the form data to the URL in the form of a query string.
- So when you hit submit, it sends the data to a backend server you specify using the method you specify. You can specify it like this:

```HTML
<form action="submit_form.php" method="POST">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" required>
  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required>
  <input type="submit" value="Submit">
</form>
```

- In this example, the form data is sent to the "submit_form.php" file on the server when the user clicks the "Submit" button
- The form data is sent using the HTTP POST method

CSS

- All of the stuff we did until now looked like garbage. We use CSS to make this garbage look better.
- A common analogy to explain HTML and CSS is that the HTML of a website is the skeleton of the website. The Javascript describes the functionality of a website - it is the brain and muscles muscles of your website. CSS makes all of this look pretty - the skin of your website.



- Every CSS file consists of a bunch of *Rules*, A template for a *rule* looks like this:

```css
/* below is a rule */
selector {
 properties
  ...
}

/* below is another rule */
another_selector {
 properties
  ...
}

/*A CSS File consists of a bunch of rules this way*/
```

- The *selector* here is used to select an element to apply those styles to. In order to help with selecting, there are two utilities provided in html - a *class* and an *ID*

A Class

- These are names you'd add to your html element. This is how you specify them in html:

```HTML
<h1 class = "this_is_a_class_name">Heading Text</h1>
```

- You can have multiple classnames for a single element. You just have to separate those classnames with a space like so:

```HTML
<h1 class = "this_is_a_class_name this_is_another">Heading Text</h1>
```

ID

- An ID is similar to class, but it's more targeted
- What I mean by 'targeted' is that a single html element can have only one ID. An ID must also be associated by a single element only.
- An ID would be exclusive to only one html element.

How Do You Put HTML and CSS Together?

- There are 3 ways to do this:
  - Inline styles
  - Internal Stylesheet
  - External Stylesheet

Inline Styles

- These are the styles you mention *in the same line* as your HTML element, in your HTML document itself.
- There is no need for selectors in in-line styles since these styles are only applied to the element you're specifying it against.
- For example:

```HTML
<div style="font-size: 69pt; color: blue;">This is your element</div>
```

- You can apply multiple styles to an inline style and they must be separated by a semicolon.

Internal Stylesheet

- This is used when you want styles to be uniform across a single html document, or a single html page.
- You enclose your CSS styles in your HTML document by enclosing it with the `<style>` tag in the `<head>` section.
- Selectors are necessary since you have to specify which element you want to apply the styles to. Here's an example:

HTML

```
<!DOCTYPE html>
<html lang="en">

<head>
  <title>Some Title</title>
  <style>
    h1 {
      color: blue;
    }

    h2 {
      color: red;
    }

    p {
      font-size: 24pt;
    }
  </style>
</head>

<body>
  <h1>
    This is a heading 1
  </h1>
  <h2>
    This is heading 2
  </h2>

  <p class="some_other_class">
    Lorem ipsum dolor sit amet, officia excepteur ex fugiat
reprehenderit enim labore culpa
    sint ad nisi Lorem pariatur mollit ex esse exercitation amet.
Nisi anim cupidatat excepteur officia. Reprehenderit
    nostrud nostrud ipsum Lorem est aliquip amet voluptate
```

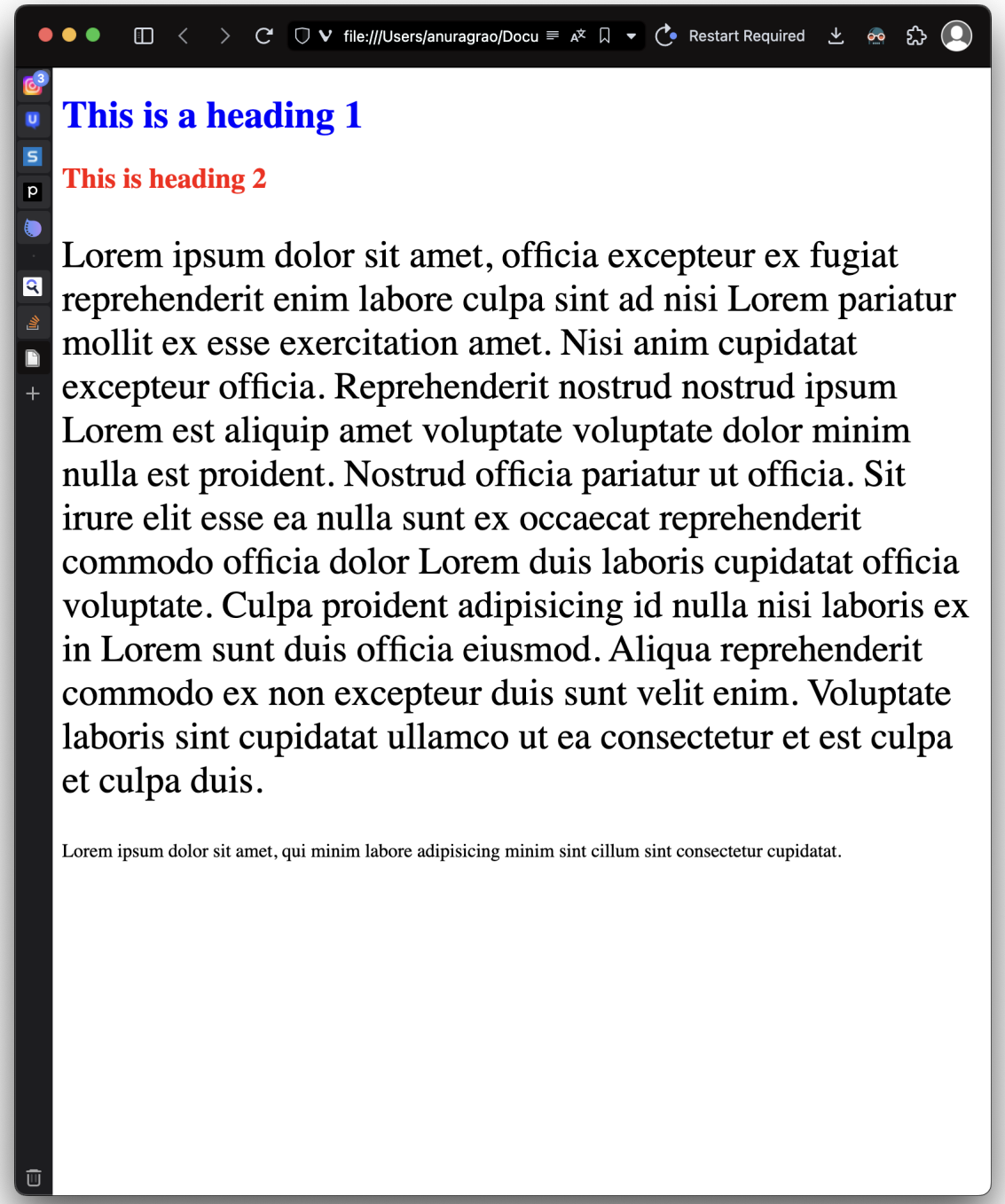
```
voluptate dolor minim nulla est proident. Nostrud officia
    pariatur ut officia. Sit irure elit esse ea nulla sunt ex
occaecat reprehenderit commodo officia dolor Lorem duis
    laboris cupidatat officia voluptate. Culpa proident
adipisicing id nulla nisi laboris ex in Lorem sunt duis officia
    eiusmod. Aliqua reprehenderit commodo ex non excepteur duis
sunt velit enim. Voluptate laboris sint cupidatat
    ullamco ut ea consectetur et est culpa et culpa duis.
  </p>


  <div class="some_class">
    Lorem ipsum dolor sit amet, qui minim labore adipisicing minim
sint cillum sint consectetur cupidatat.
  </div>
</body>

</html>
```

- Here, I applied the style `color: blue` and `color: red` for all `h1`s and all `h2`s respective. All `p` tags are also given a `font-size` of `24pt`

Info

A `div` tag is a generic tag. `div` stands for division and it's used to make different boxes, aka divisions in your webpage. More about this in later sections, specifically under the *box model* section.

- This is how the above example renders on the website:



Info

Lorem ipsum is a convention in web development, programming in general really. It is used to fill in dummy text. This looks good enough to look like actual text to get to know how real text would look like. The words itself don't make any sense but you get to know how your text would look like.

- Now, you might not want to target all the `p` tags but only a specific number of `p` tags.
- If you see a normal website, different elements are styled differently.

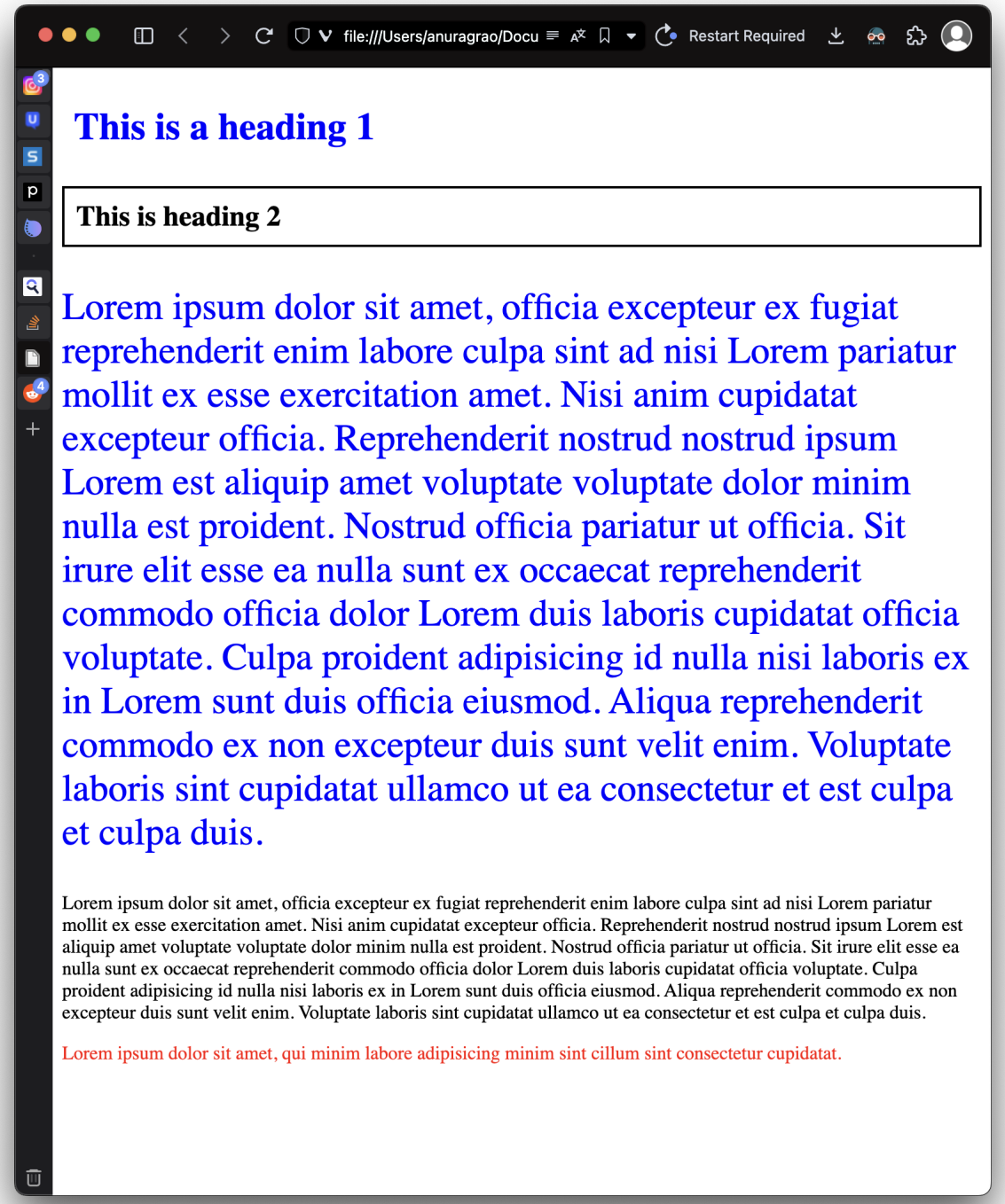
- This is where classes and IDs come in.
- Here's an example:

HTML

```
<!DOCTYPE html>
<html lang="en">

<head>
  <title>Some Title</title>
  <style>
    .text_blue {
      color: blue;
    }

    .padding10 {
      padding: 10px;
    }

    p.text_blue {
      font-size: 24pt;
    }

    .text_red {
      color: red;
    }

    #this_is_h2_pa {
      border: 2px solid;
    }
  </style>
</head>

<body>
  <h1 class="text_blue padding10">
    This is a heading 1
  </h1>
  <h2 class="padding10" id="this_is_h2_pa">
    This is heading 2
  </h2>

  <p class="text_blue">
    Lorem ipsum dolor sit amet, officia excepteur ex fugiat
reprehenderit enim labore culpa
    sint ad nisi Lorem pariatur mollit ex esse exercitation amet.
```

```
Nisi anim cupidatat excepteur officia. Reprehenderit
    nostrud nostrud ipsum Lorem est aliquip amet voluptate
voluptate dolor minim nulla est proident. Nostrud officia
    pariatur ut officia. Sit irure elit esse ea nulla sunt ex
occaecat reprehenderit commodo officia dolor Lorem duis
    laboris cupidatat officia voluptate. Culpa proident
adipisicing id nulla nisi laboris ex in Lorem sunt duis officia
    eiusmod. Aliqua reprehenderit commodo ex non excepteur duis
sunt velit enim. Voluptate laboris sint cupidatat
    ullamco ut ea consectetur et est culpa et culpa duis.
  </p>

  <p>Lorem ipsum dolor sit amet, officia excepteur ex fugiat
reprehenderit enim labore culpa sint ad nisi Lorem pariatur
    mollit ex esse exercitation amet. Nisi anim cupidatat
excepteur officia. Reprehenderit nostrud nostrud ipsum Lorem
    est aliquip amet voluptate voluptate dolor minim nulla est
proident. Nostrud officia pariatur ut officia. Sit irure
    elit esse ea nulla sunt ex occaecat reprehenderit commodo
officia dolor Lorem duis laboris cupidatat officia
    voluptate. Culpa proident adipisicing id nulla nisi laboris ex
in Lorem sunt duis officia eiusmod. Aliqua
    reprehenderit commodo ex non excepteur duis sunt velit enim.
Voluptate laboris sint cupidatat ullamco ut ea
    consectetur et est culpa et culpa duis.</p>


  <div class="text_red">
    Lorem ipsum dolor sit amet, qui minim labore adipisicing minim
sint cillum sint consectetur cupidatat.
  </div>
</body>

</html>
```

- And this is how it renders as:



- If you want to target a particular class of elements, you prepend it with a `.` in your CSS followed by the classname. For example, if I want to target all elements with the class `text_blue`, I use the selector `.text_blue`
- It's a similar idea for IDs as well, only that `.` is replaced by a `#`. So, if I have to target an ID of `this_is_h2_pa`, I use the selector `#this_is_h2_pa`
- You can also combine class selectors with other selectors. In the example above, I wanted to target only the `p` tags with `text_blue` class, and apply `font-size: 24pt` to them, so I used the selector `p.text_blue`
- Observe how this wasn't applied to the normal `p` tag
- Just go through the html and the rendered webpage and the code is pretty self explanatory. Web dev is mostly about exploring shit yourself, you can't rely

purely on notes to ace an exam. You'd know basics but it isn't possible to put everything in the notes. This pdf is prolly already long af.

External Stylesheets

- You can make a separate .css file and link it to your html, or multiple html documents
- This is usually used when you want to apply a style to multiple pages in a website. For example, you might want all buttons on your website to look similar, then you put your button styles in an external CSS file and import it in all the html files.
- For example:
  - I create a `css_demo_styles.css` inside the `css` folder
  - I put these styles in the CSS file:

```css
.text_blue {
  color: blue;
}

.padding10 {
  padding: 10px;
}

p.text_blue {
  font-size: 24pt;
}

.text_red {
  color: red;
}

#this_is_h2_pa {
  border: 2px solid;
}
```

- I then put a link to this in my HTML. To do this, you have to add this line to your `<head>` section of the HTML

```HTML
  <link rel="stylesheet" href="css/css_demo_style.css"
type="text/css">
```

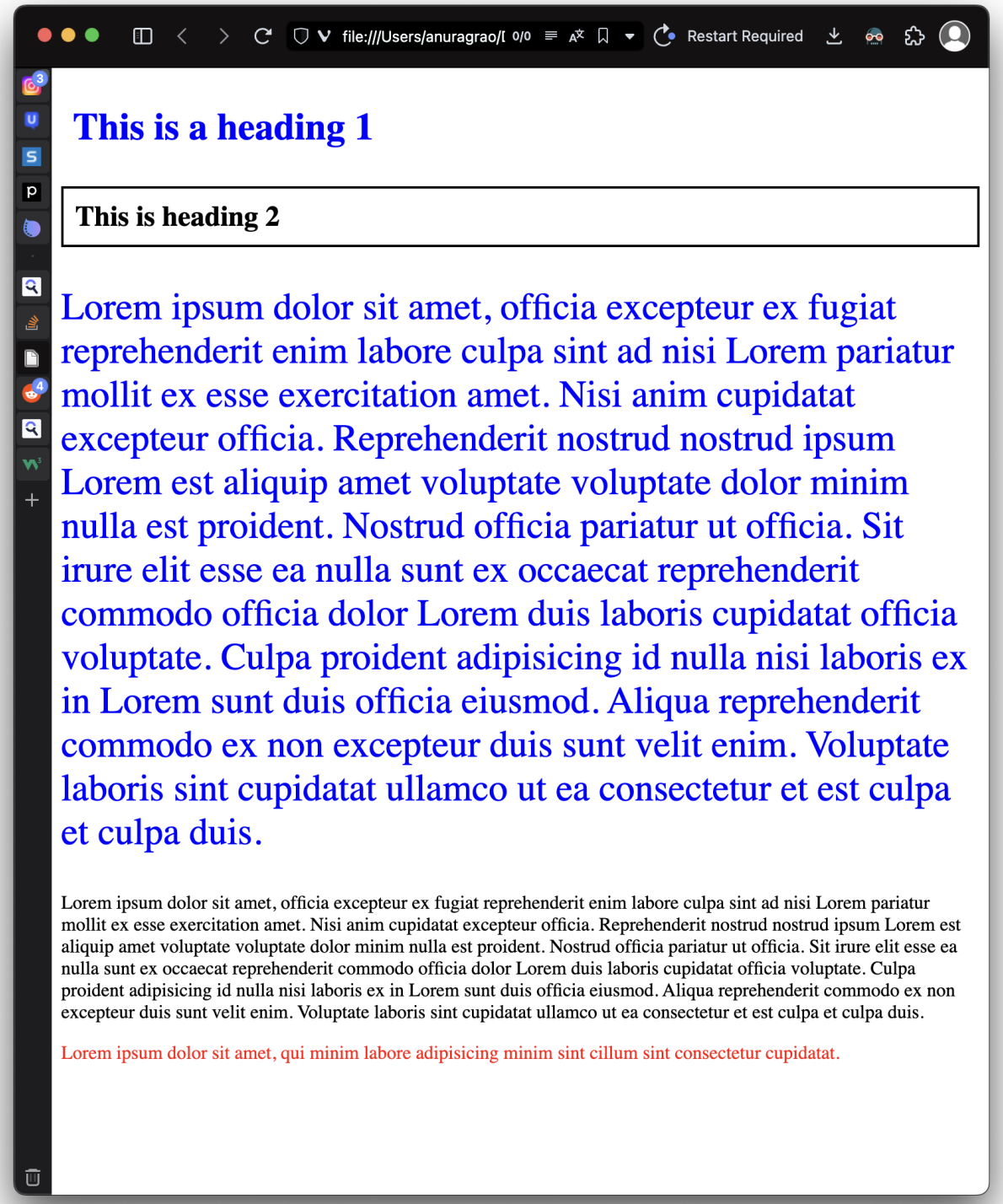
- The entire HTML doc looks like this:

HTML

```
<!DOCTYPE html>
<html lang="en">

<head>
  <title>Some Title</title>
  <!-- Notice the absence of a style tag -->
  <link rel="stylesheet" href="css/css_demo_style.css"
type="text/css">
</head>

<body>
  <h1 class="text_blue padding10">
    This is a heading 1
  </h1>
  <h2 class="padding10" id="this_is_h2_pa">
    This is heading 2
  </h2>

  <p class="text_blue">
    Lorem ipsum dolor sit amet, officia excepteur ex fugiat
reprehenderit enim labore culpa
    sint ad nisi Lorem pariatur mollit ex esse exercitation amet.
Nisi anim cupidatat excepteur officia. Reprehenderit
    nostrud nostrud ipsum Lorem est aliquip amet voluptate
voluptate dolor minim nulla est proident. Nostrud officia
    pariatur ut officia. Sit irure elit esse ea nulla sunt ex
occaecat reprehenderit commodo officia dolor Lorem duis
    laboris cupidatat officia voluptate. Culpa proident
adipisicing id nulla nisi laboris ex in Lorem sunt duis officia
    eiusmod. Aliqua reprehenderit commodo ex non excepteur duis
sunt velit enim. Voluptate laboris sint cupidatat
    ullamco ut ea consectetur et est culpa et culpa duis.
  </p>

  <p>Lorem ipsum dolor sit amet, officia excepteur ex fugiat
reprehenderit enim labore culpa sint ad nisi Lorem pariatur
    mollit ex esse exercitation amet. Nisi anim cupidatat
excepteur officia. Reprehenderit nostrud nostrud ipsum Lorem
    est aliquip amet voluptate voluptate dolor minim nulla est
proident. Nostrud officia pariatur ut officia. Sit irure
    elit esse ea nulla sunt ex occaecat reprehenderit commodo
officia dolor Lorem duis laboris cupidatat officia
```

```
    voluptate. Culpa proident adipisicing id nulla nisi laboris ex
in Lorem sunt duis officia eiusmod. Aliqua
    reprehenderit commodo ex non excepteur duis sunt velit enim.
Voluptate laboris sint cupidatat ullamco ut ea
    consectetur et est culpa et culpa duis.</p>


  <div class="text_red">
    Lorem ipsum dolor sit amet, qui minim labore adipisicing minim
sint cillum sint consectetur cupidatat.
  </div>
</body>

</html>
```

And the page renders like this:



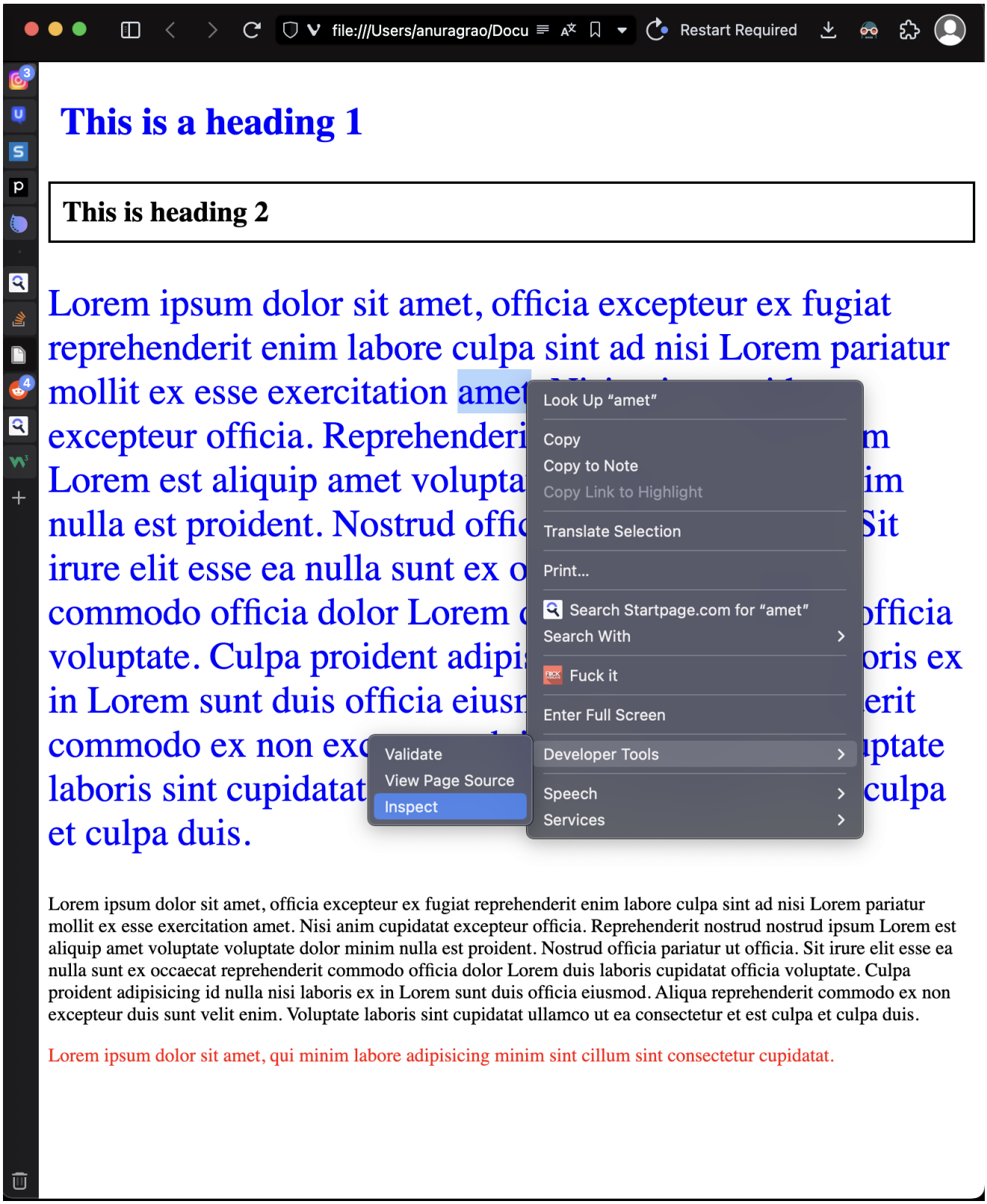
Conflicts

- When you can put CSS in 3 different places, and combine these ways as well, there would be conflicts
- By a *conflict*, I mean that an element is supposed to have `color: blue` according to the external style sheet, `color: red` according to the internal style sheet, and `color: black` according to the inline style.
- In these cases, the in-line is given higher precedence, then the internal style sheet, and then the external style sheet. So the element in this case will have `color: black`

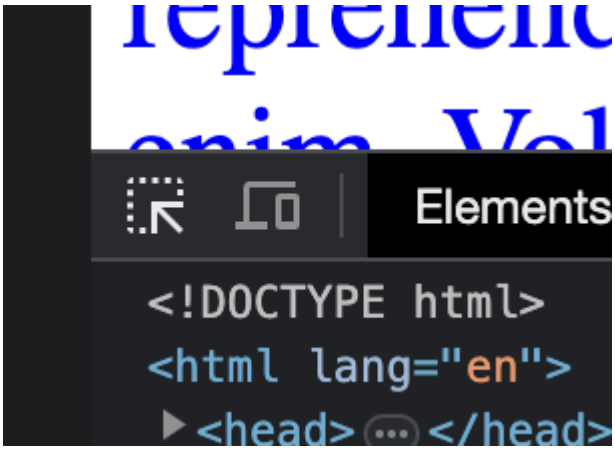
- You can override this behaviour by specifying `!important` next to the CSS rule. For example, if my external style sheet consisted of the rule `color: blue !important` for a particular element, it would override the internal and inline styles.
- However, this is not usually recommended as it makes debugging super super hard. You might have your heading in red colour due to some random external style sheet rule and you'd be left having no clue why it's red.

The Box Model

- Every HTML element is treated as a box. This can be visualised in the browser dev tools.
- If you right click on the browser and hit inspect:

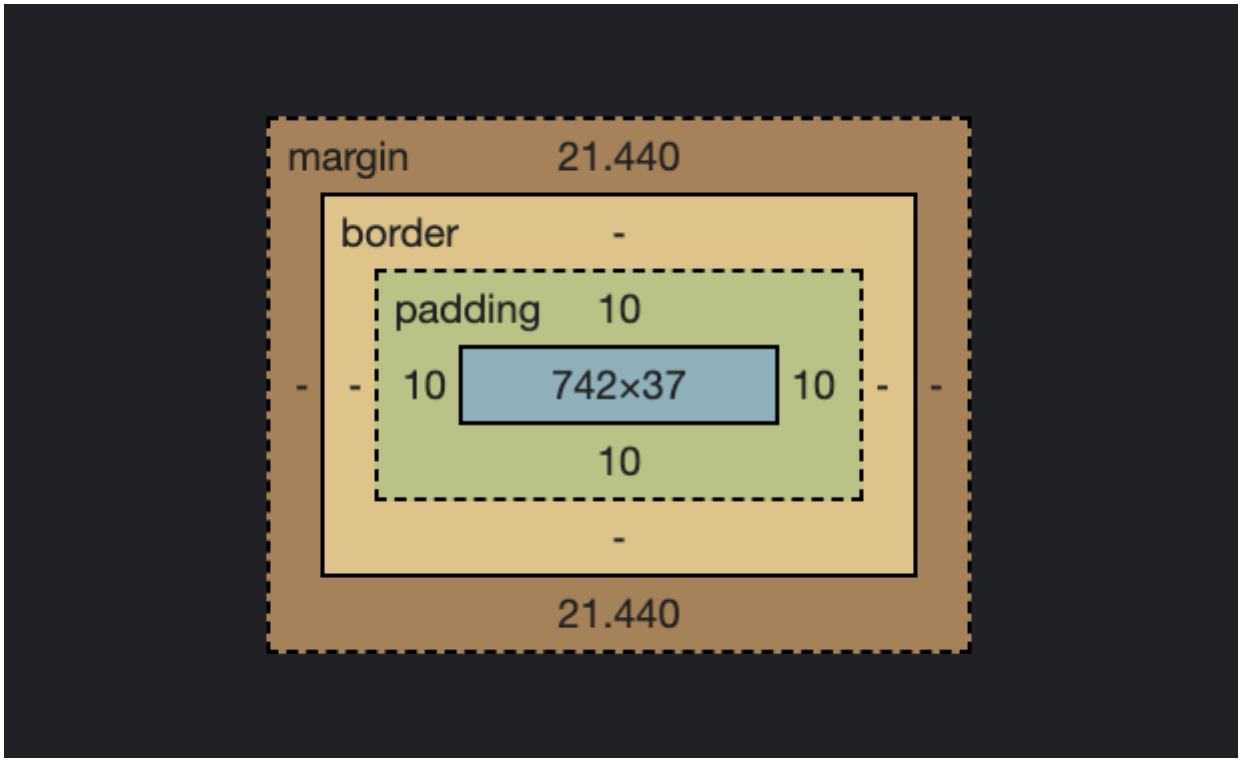


And use this button to select our first heading element:

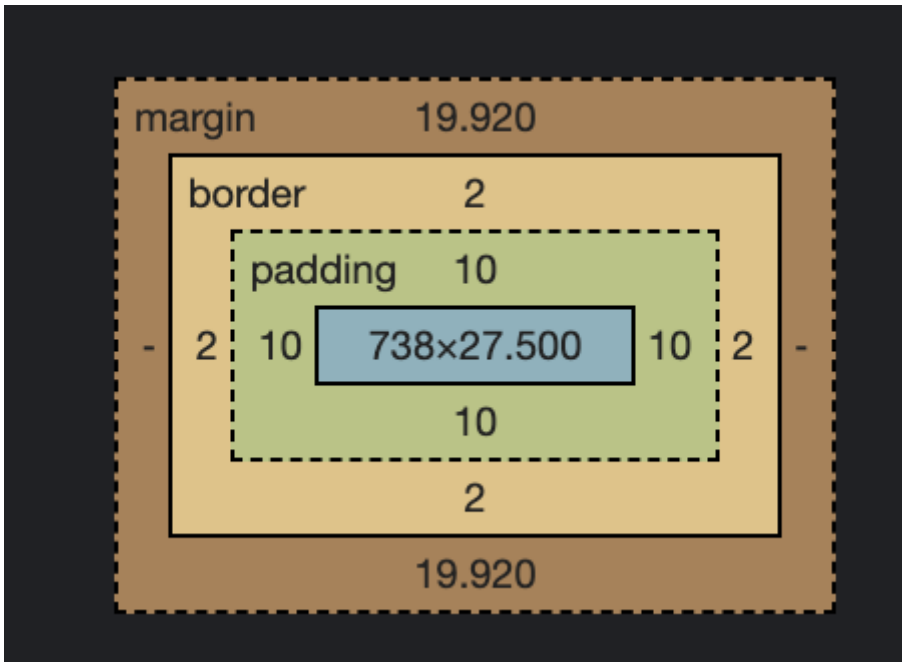


(the button on the top left)

Scroll down a little bit in the styles tab to find the box model for that element:



- This is called the *box model*. Our html element `h1` consists of the *content*, *padding*, *border*, and *margin*
- The content is the content of the element. Pretty self explanatory
- The padding defines some spacing for the content till the edge of the box
- Then there is the border, all border widths come here. If I select the second element, that is `h2` with a border, it looks like this:



- At the end there are margins. This space comes around the border, and doesn't appear as if it's a part of your element.

Positioning These Boxes

- The *position* attribute in CSS can be used to define where these boxes are.
- The position attribute can have one of these values:

Value	Description
static	Default value. Elements render in order, as they appear in the document flow
absolute	The element is positioned relative to its first positioned (not static) ancestor element
fixed	The element is positioned relative to the browser window
relative	The element is positioned relative to its normal position, so "left:20px" adds 20 pixels to the element's LEFT position
sticky	The element is positioned based on the user's scroll position A sticky element toggles between `relative` and `fixed`, depending on the scroll position. It is positioned relative until a given offset position is met in the viewport - then it "sticks" in place (like position:fixed). **Note:** Not supported in IE/Edge 15 or earlier. Supported in Safari from version 6.1 with a -webkit- prefix.
initial	Sets this property to its default value. Read about *initial*
inherit	Inherits this property from its parent element. Read about *inherit*

- For example:

```css
#parent1 {
  position: static;
  border: 1px solid blue;
  width: 300px;
  height: 100px;
}
#child1 {
  position: absolute;
  border: 1px solid red;
  top: 70px;
  right: 15px;
}

#parent2 {
  position: relative;
```

```
  border: 1px solid blue;
  width: 300px;
  height: 100px;
}

#child2 {
  position: absolute;
  border: 1px solid red;
  top: 70px;
  right: 15px;
}
```

- You can't rely on my notes for figuring these things out and what element does what. You have to mess around with CSS yourself and find out what each style does. There are an infinite number of styles and combinations. You can only know about them from experience and/or googling. Nobody knows about all of the styles present in CSS.

Javascript

- This is used to build some action into your websites.
- The stuff we built until now was all static and there was no logic to it. For example, in the forms, nothing really happened when you hit submit. The data went nowhere. There was no logic to it.
- You can use a language called Javascript to put logic into your sites.
- Every browser has a javascript engine. This JS engine is supposed to run javscript in the browser. It is like a compiler. Javascript is a JIT compiled language. JIT stands for *just in time*
- Every line is compiled just before executing. This has the advantages of an interpreted language like Python. But also has the advantage of compiled languages like C.
- For example, if you have a function that is repeatedly called, it has to be compiled only the first time it's called. All subsequent calls do not have to spend time compiling the same block again.
- Now if you look at this from a cyber security guy's point of view, you're essentially allowing a website, to run code, on your computer. This website could be malicious and run whatever it wants on your computer. This is one of the disadvantages of JS.
- To protect against this, browsers do implement sandboxing of any JS run in the browser. This isn't explained a lot on the slides so I won't waste your time. You can search online how browser sandboxing works if you're into it.

How Do You Put Javascript In Your Site?

- You can use one of these two methods:

1. Use a `<script>` tag and write JS inside it in your HTML doc. Eg:

```HTML
<script>
// some js here
</script>
```

3. Use a `<script>` tag with an src reference to an external script file. Eg:

```HTML
<script src="myScript.js"></script>
```

- You can put this `<script>` tag in either the head or body section.
- When you put a `<script>` tag in the head section, it is run before the HTML site loads into your browser.
- When you put it in your `body` section, it is run as your page is being rendered
- Putting js in a separate file is advantageous because it makes your code more organised and easier to read.

Running JS

- The stuff you put in your page automatically runs when your page is loaded.
- You can see all console logs if you right click -> inspect, and then go to the console tab.

The Syntaxes And Nuances Of JS

- Javascript is a weird language. It's a dumpster fire.
- The web as a whole only is a dumpster fire. Nothing is organised and things rarely make sense. But it works. Somehow. Nobody knows how.

Comments

- You can create comments in JS using a similar syntax to C.
- `//` for single line comments and `/* for multiline comments */`

Semicolons

- This is nice about Javascript. It isn't necessary to use semicolons in Javascript
- It automatically inserts semicolons at the end of a line.
- If you want to write more than one statement in one line, you have to put a semicolon though.
- So, people who like semicolons can put semicolons. People who don't (I'm looking at you Python devs) don't have to.

Variables

- Variables do not have to have an explicit type. They are implicitly taken. This is similar to Python, and *unlike* C.
- You can declare a variable in 3 ways:

1. `var`
2. `let`
3. `const`
4. Use without declaring

Var

When you declare a variable with `var` like:

```JS
var some_variable = 10;
```

The scope of this variable is limited to the function.

- This function is the main function if it's declared outside any specific function.
- You can reassign and redeclare this variable as you want

```JS
var some_variable = 10;
var some_variable = 20; // still valid!
some_variable = 69; // still works
```

- Now why you'd want to redeclare it again - I have no idea. Javascript just allows you to. Javascript allows you to do whatever you want in life.

Let

- The `let` way of declaring variables is more preferred in modern javascript. Why? I'll tell you.
- Variables declared using the `let` keyword are limited to their block. A block is defined by a `{}` curly brace. So variables declared inside `if`, `while`, `for` blocks don't interfere with things outside and can't be accessed outside that block. This leads to less variable conflict.
- And also, these *can't* be redeclared. Why would you want to??
- Can be reassigned though. I mean- They are *vari*ables. They *vary*. What else do you expect them to do.

```JS
let some_variable = 10; // nice
let some_variable = 20; // not nice. won't work
some_variable = 30; // works. nice
```

Const

- These people are the imposters. They think they are variables but they are not. They are constants.
- Like `let`, they too have block scope, but since they are constants, they can't be reassigned or redeclared

```JS
const some_variable = 10; // nice
const some_variable = 20; // not nice. won't work
some_variable = 30; // not nice. also won't work
```

Global

- What if you ditch all of this and decide to be a rebel and don't declare at all? JS doesn't care. It lets you do what you want in life

```JS
some_variable = 69; // no problem
```

- This let's you redeclare, reassign and do whatever you want.
- Redeclare and reassign are the same thing here really. You're not declaring only- how will you *re*declare.
- The scope of this variable will be global. Any function and anything can access this variable. Not recommended as this can cause variable name conflicts.

Here's a summary

Keyword	Scope	Can be reassigned?	Can be redeclared?
`var`	function	yes	yes
`let`	block	yes	nope
`const`	block	nope	nope
`global` (without declaration)	global	yes	yes

Datatypes

- You can declare variables with `var` `let` and all but what *datatype* are they?
- JS doesn't care.
- It implicitly assumes a datatype based on the context
- There are a few primitive and few non primitive ones.
- Primitive:

1. `number`
2. `boolean`
3. `string`
4. `undefined`: when variable is declared but not defined
5. `null`: intentional absence of a value

- Non primitive:

1. `Number`
2. `Object`
3. `Date`
4. `String`
5. `Object`
6. `Boolean`

- Observe how a number can be a primitive datatype an a non primitive object. By convention, objects always start with an uppercase. So, a primitive number is just a `number` but a number object is a `Number`
- The `Number` object consists of various methods to perform operations on the number, like `toFixed()` etc.
- When you try to perform operations on a primitive number, it's implicitly temporarily converted to a `Number` object.
- For example:

```JS
let num1 = new Number(20);
let num2 = 20;

console.log(typeof num1); // Output: object
console.log(typeof num2); // Output: number

console.log(num1.toFixed(2)); // Output: 20.00
console.log(num2.toFixed(2)); // Output: 20.00

console.log(typeof num1); // Output: object
console.log(typeof num2); // Output: number
```

- Also see how there is no `float` or `int` like in C. A number is a number.

Objects

- We said `Number` is an *object*, but what on earth is an object.
- According to JS, almost everything is an object.
- Arrays are objects, Functions are objects, Regular Expressions are objects, Dates are objects, Strings are objects, I am an object, you are an object

- An object is just a key value pair. A glorified Python dictionary, that can hold multiple nested values.

Ways To Make Objects

1. Object Literal

```JS
let some_object = {
  var1: "bun samosa",
  "var2": "bun samosa but string",
  var3: "masala puri",
};
```

- This is just defining key value pairs.
- You can access these key value pairs using two methods.
- One of which is this:

```JS
console.log(some_object.var3); // masala puri
```

- But if your `key` is a string, like it is in `"var2"`, then you can't use this syntax as `some_object."var2"` doesn't make sense. In that case, you can use the `[]` syntax:

```JS
console.log(some_object["var2"]); // bun samosa but string
```

2. Using A Constructor

```JS
function Person(name, age) {
 this.name = name;
 this.age = age;
}

function Apartment(name, age){
 this.name = name;
 this.age = age;
}
```

- A constructor is just a fancy function that's used to make a new object
- The `this` keyword refers to the object being created. It is analogous to `self` in Python. It's a placeholder for the instance of the object.
- You can also add methods(aka functions) to the constructor's prototype:

```JS
function Person(name, age) {
 this.name = name;
 this.age = age;
 this.sayName = function (){
  console.log(this.name);
 };
}
```

- If this method is common across all instances of an object, you can avoid duplication of functions by using the `prototype` object.
- A `prototype` object is a shared object that is used by all instances of the constructor. Adding methods to the `prototype` can help conserve memory and helps with the JIT compiler that I talked about during the introduction.
- You can add the sayName function as a prototype like this:

```JS
Person.prototype.sayName = function (){
 console.log(this.name);
}
```

- Finally, to create an object using this constructor, use the `new` keyword:

```JS
person1 = new Person("Anurag", 18); // 'this' will be person1
person2 = new Person("Cupcake", 5); // 'this' will be person2

person1.sayName(); // Anurag
person2.sayName(); // Cupcake
```

Here's the entire code for reference. I have used the shared `prototype` object to create the sayName function:

```JS
function Person(name, age) {
  this.name = name;
  this.age = age;
}

Person.prototype.sayName = function () {
  console.log(this.name);
}


let person1 = new Person("Anurag", 18);
let person2 = new Person("Cupcake", 5);

person1.sayName(); // Anurag
person2.sayName(); // Cupcake
```

3. Using A `class`

- You can also use the class syntax. Here's the same object as above but in the form of a class:

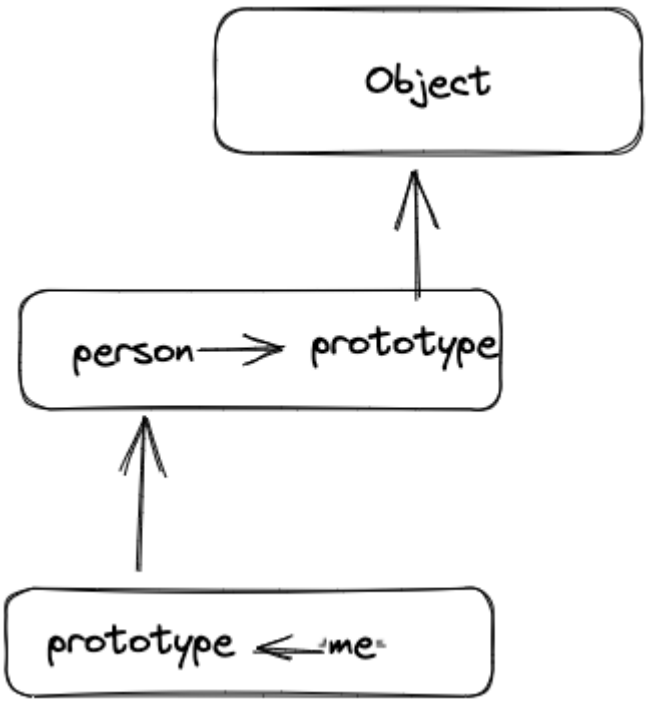
```JS
class Person {
    constructor(name, age) {
        this.name = name;
        this.age = age;
    }
    sayName() {
        console.log(this.name);
    }
}
```

```
let person1 = new Person("Anurag", 18);
let person2 = new Person("Cupcake", 5);

person1.sayName(); // Anurag
person2.sayName(); // Cupcake
```

> **Info**
>
> `console.log` is how you do print statements in JS

Prototypal Inheritance

- Note that this is different from normal inheritance. This is *prototypal* inheritance.
- Inheritance in JS is implemented differently than traditional languages. So throw any knowledge you have about inheritance out the window.
- Every object in JS has this *prototype* object. This is a common object for every instance of that object.
- We use this *prototype* object to implement inheritance.
- This is how we do it: *You set the prototype of the child object to be the parent object*, and so on. This creates a chain when you inherit from multiple ancestors. This chain is called a prototype chain.



- Every object in JS inherits from the main `Object` object. The parent object is called `Object`. This is what we use when we access properties like `Object.call()` and `Object.create()` etc.
- You can see this better in a console.log. I will show that in the example
- There are two ways to set this prototype to be the parent object:

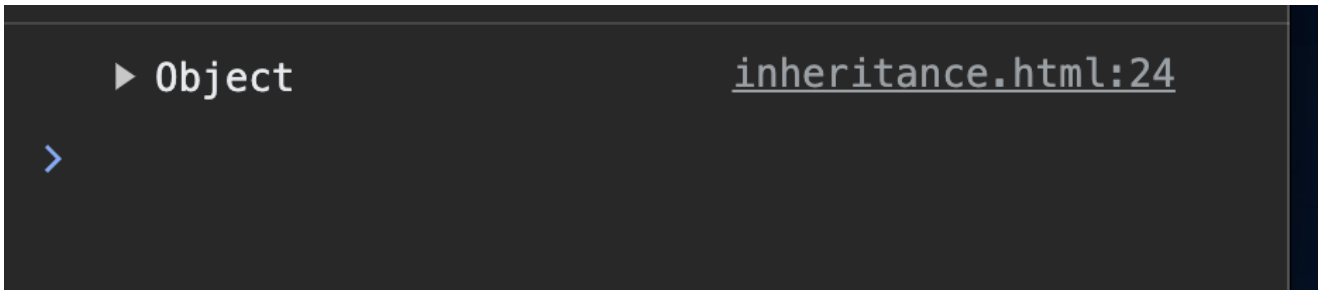
1. `Object.create()`
2. Constructor Method

`Object.create()`

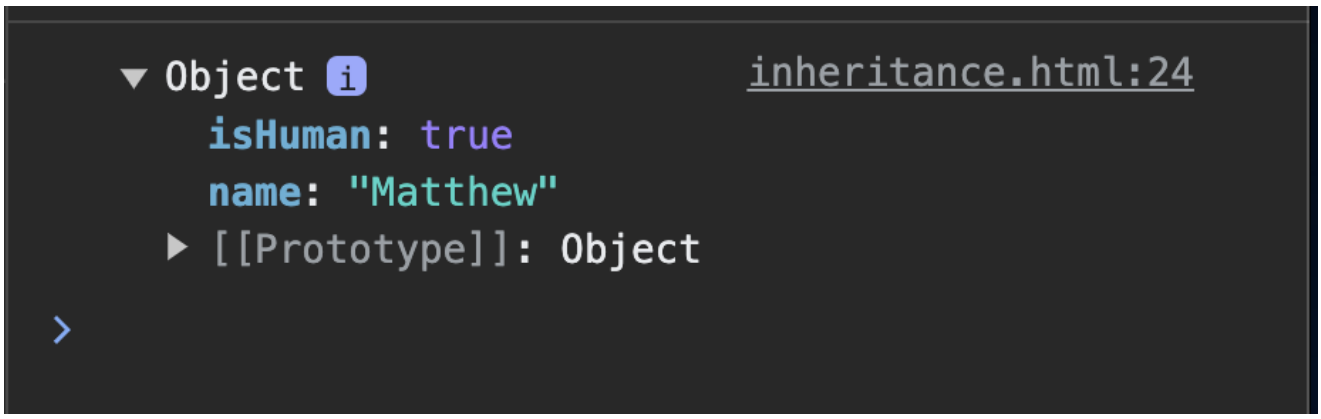
- `Object.create()` is a method that creates a new object.
- It sets the prototype of the new object to the object that was passed as a parameter.
- It's a way to create an object that inherits properties and methods from another object.

```JS
const person = {
  isHuman: false,
  printIntroduction: function () {
    console.log(`My name is ${this.name}. Am I human?
${this.isHuman}`);
  },
};


const me = Object.create(person);

me.name = 'Matthew'; // "name" is a property set on "me", but not
on "person"
me.isHuman = true; // Inherited properties can be overwritten

me.printIntroduction();
// Expected output: "My name is Matthew. Am I human? true"
```

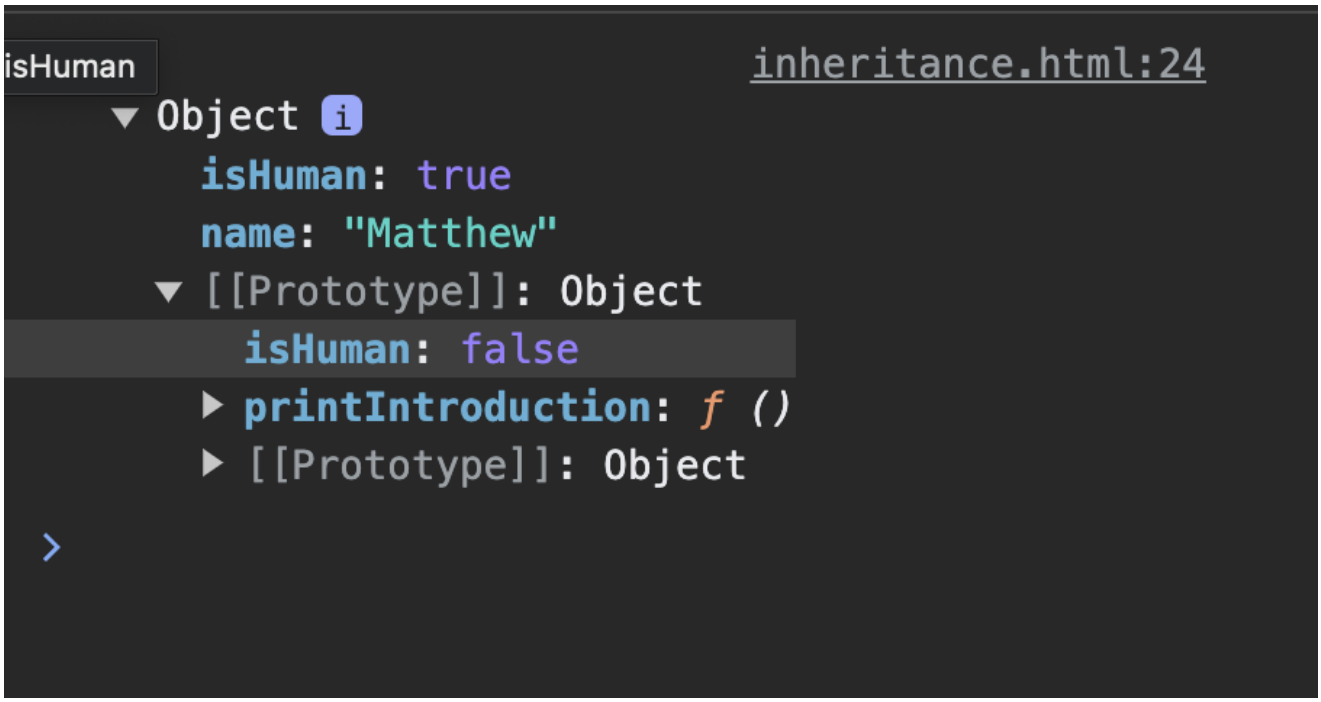
- Here's the `console.log(me)`. This shows all the properties of the object in the browser:



If I click the drop down:



If I expand the `[[prototype]]`:



Observe how the prototype is the parent object `person`
Now if I expand the prototype of this:

- This is the main parent `Object` object. Every object in JS is a descendant of the `Object`. It's kinda like the Genghis khan of JS Objects
- You'd have also seen that there are two `isHuman` properties. One in the child object and one in it's parent.
- Whenever we try to access a property or method of an object, it first searches for it in the object itself, if not found, it searches in it's prototype, if not found, it then

searches for it in it's prototype's prototype. This chain goes on until the prototype reaches NULL.

Constructor Method

- This is a little more manual method than the previous one.
- In real life JS, we don't use any of these methods. We just use `class child extends parent{ // properties }` and we are done. But this is PES we are talking about-
- Here's how you use the constructor method to do inheritance:

1. **Define the base object constructor**:

```JS
function Vehicle(type, model, color) {
  this.type = type;
  this.model = model;
  this.color = color;
}
Vehicle.prototype.drive = function() {
 console.log("Driving a " + this.type);
}
```

2. **Define the derived object constructor which in turn calls the base object constructor**: A derived object (or a subclass) can call the base object's constructor using the `call()` method. For instance, a `Car` object can call the `Vehicle` constructor like this:

```JS
function Car(model, color) {
  Vehicle.call(this, 'Car', model, color); // 'Car', model, color are parameters to pass to the Vehicle constructor
}
```

The `call()` method calls the `Vehicle` constructor with the context of `this` (the `Car` object) and passes the arguments `'Car'`, `model`, and `color`

3. **Assign the prototype of derived object as the base object**: This is done so that the derived object can access the base object's properties and methods:

```JS
Car.prototype = Object.create(Vehicle.prototype);
// Car.prototype.prototype = Vehicle.prototype
```

In the example, `Object.create(Vehicle.prototype)` creates a new object with the prototype set to `Vehicle.prototype`. This new object is then assigned to `Car.prototype`. (why would you do this when you can just do object.create with the *actual* object only 🤦)

4. **Assign the prototype.constructor to derived object constructor**: This is done to ensure that the constructor property of instances of the derived object points back to the derived constructor, not the base constructor. If this step is skipped, instances of `Car` would have a constructor of `Vehicle`:

```js
Car.prototype.constructor = Car;
```

- Now `Car` can use the properties and methods of vehicle:

```js
let Ciaz = new Car("Ciaz", "Grey");
Ciaz.drive(); // uses the Vehicle.prototype.drive() method
// Output: Driving a Car
```

Loops And If Constructs

- JS supports the normal `if`, `else if`, `while`, `do while` etc.
- One special thing to mention is the `for/in` loop
- This is kinda like Python - The `for(x in y)` loop iterates through `y`.
- For example:

```js
let some_object = {
  first_name: "Anurag",
  last_name: "Rao",
  age: 18,
};

for (key in some_object) {
  console.log(key);
  console.log(some_object[key]);
}

// output:
// first_name
// Anurag
// last_name
// Rao
```

```
// age
// 18
```

- In the above example, `for(x in y)`, where `x` was the `key` and `y` was `some_object`, we iterated through the keys, and also printed their corresponding values.
- Now, arrays in JS are also an object, you can think of them as a special kind of object with the `keys` being their indices, and the value being the value at those indices. So:

```JS
let arr = [1,2,3];
// is kinda equivalent to:
let objarr = {
0: 1,
1: 2,
2: 3,
}
```

So when you do:

```JS
for(const i in arr){
 console.log(i);
}
// it will print the indices:
// 0
// 1
// 2
```

To print the values itself, you can either do `arr[i]` or use a `for...of` loop:

```JS
for(const i of arr){
 console.log(i);
}
// prints
// 1
// 2
// 3
```

Info

> If you want to print multiple values in a single console.log expression, you gotta do string concatenation.
> `console.log("some string", "some other string");` will not work like it did in Python. We gotta do `console.log("some string" + "some other string");` and this will concatenate these values

Equality and Strict Equality

- JS, unlike other languages has two different kinds of equality. `==` and `===`
- `==` is equality
- `===` is strict equality
- This bullshit leads to situations like this:

- When you `==` compare two values, it tries to change the operands being compared to the most appropriate datatype implicitly.

- So when we compared `"0" == 0`, it returns `true` because it converted the number `0` to a string `"0"`
- The same thing happened when we did `0 == []`. It converted `[]` to a number `0` since it's empty.
- But when you try to convert `[]` to an string, it becomes an empty string `""` which is not the same as `"0"`, so it returns false.
- This is a 100% coming in the ISA to haunt us all, so be careful.
- To solve this messed up equality, we use `===`, that is strict equivalence.
- This does not do any type coercion that `==` did and just compared them for what they are.
- So `"0" === 0` will return false, `"0" === []` will return false and so will `0 === []`
- That's why you'd see `===` used for equality in most places in JS. It makes more sense and helps avoid unintuitive behaviour. We programmers are simple creatures. Complex shit like this is too much effort.
- You can also do strict in-equivalence like this: `!==`

String Concatenation and Number Unintuitive Behaviour

- See the below examples:

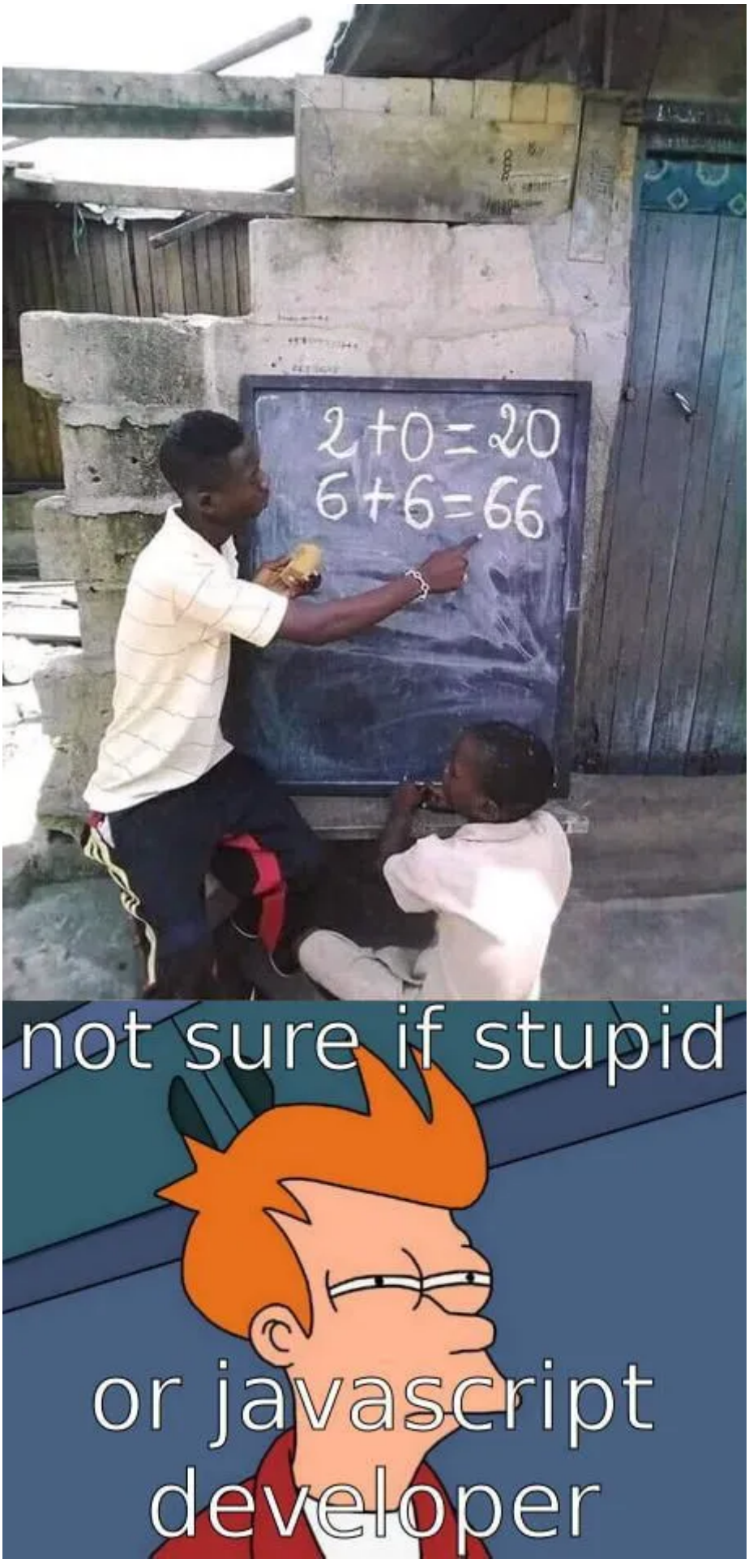
```JS
let text1 = "ABC"
let text2 = "DEF"


console.log(text1 + text2); // ABCDEF - works intuitively
console.log(text1 + 2 + 2); // ABC22 - wtf
console.log(2 + 2 + text1) // 4ABC - wtf again
```

- Here's what's happening. When we concatenated 2 strings, it works as expected.
- When we have a string first and then 2 numbers, it treated the numbers as strings too and concatenated them instead of adding
- In the third case, we gave the numbers first. JS hasn't seen the string yet so it added the two numbers. When it came to the string, it then switched to concatenation mode and concatenated `4` and `ABC`
- It also leads to situations like this:

```JS
console.log("5" + 5); // 55
console.log(5 + 5) // 10
console.log("11" + 1) // 111 because 1 is converted to string
```

```
console.log("11" - 1) // 10 because 11 is converted to int (can't
negate strings, only add, aka concatenation)
```

- Be careful with this too in the ISA. These kinds of question would exist only to get the average down.



Arrays

- Just like in Python, arrays are defined with `[]`

- *They can be heterogeneous* just like in Python, unlike C.
- You can also make an array using the `new Array()` syntax. `Array()` is a constructor and we use the `new` keyword on it (see the 'objects' section above)

```JS
arr1 = [1, 2, 3, 4]; // object literal method
arr2 = new Array(69); // object constructor method. Creates an
array of size 69
arr3 = new Array(5, 6, 7); // object constructor method, but
creates an array with elements 5, 6, and 7


console.log(arr1);
console.log(arr2);
console.log(arr3);

// output
// [ 1, 2, 3, 4 ]
// [ <69 empty items> ]
// [ 5, 6, 7 ]
```

- You can get the length of an array using the `.length` property of the array:
- Like this:

```JS
console.log(arr1.length); // 4
```

- Note that you can have `holes` in your array. These are undefined values in between the array.
- Since you can have holes in an array, and the length of the array can be modified at runtime, the length of the array isn't really meaningful in finding out how many elements are there in the array.

```JS
arr2[2] = 40;
console.log(arr2); // [ <2 empty items>, 40, <66 empty items> ]
```

- You can also set the length of the array manually during runtime:

```JS
console.log(arr1.length); // 4
arr1.length = 10;
console.log(arr1); // [ 1, 2, 3, 4, <6 empty items> ]
console.log(arr1.length); // 10
```

```
// notice how length is 10 even though there are only 4 defined elements
```

Info

If you use a `for(i in arr1)` loop, this behaves differently than Python. It gives you the indices of the array. This is generally not recommended because it's not really useful. If you want the behaviour we expected from Python, we either use a `for(i of arr1)` or the `.forEach()` method. There are many other ways to achieve the same thing - JS being JS. This isn't mentioned in the slides so I'm not gonna go into depth about them.

JS

```
for (const i in arr1) {
  console.log(typeof i + i);
}
// output
// string0
// string1
// string2
// string3
```

Think about why I used `const i` instead of just `i`. If you don't get it immediately, check out the variables section upstairs, specifically the different types of declaring variables

- Since arrays can be heterogenous, they can have other objects inside them. A function can also be an array element. An array itself is a special kind of object really, with the keys being the indices, and the values are the values of the array
- There are a few array functions you can perform using build in functions:

1. `.push` - push an element to the end of the array
2. `.pop` - pop an element from the end of the array
3. `.shift` - delete element at the front of the array, *shifting* the other element's indices to the left
4. `.unshift` - inserting element at the front of the array, *unshifting* their indices
5. `.join` - join elements of an array
6. `.indexOf` - returns the index of an element in an array
7. `.sort` - sorting an array, ascending by default
8. `.concat` - concatenate two arrays
9. `.slice` - slicing a section from the array (to get a subset of the array)

10. `.splice` - modifying the elements of an array. You can add, delete, modify at any index with this method

.push()

- This method is used to push an element to the back of the array. Here's how it works:

```JS
arr1 = [1, 2, 3, 4];

console.log(arr1); // [ 1, 2, 3, 4 ]
arr1.push(5); // pushes 5
console.log(arr1); // [ 1, 2, 3, 4, 5]
arr1[arr1.length] = 6; // this will also do the same job
console.log(arr1); // [ 1, 2, 3, 4, 5, 6]


arr2 = [5, 6];
console.log(arr2); // [ 5, 6 ]
arr2.length = 10; // introducing holes into the array
arr2[arr2.length] = 7; // pushes at the end
// even if you do arr2.push(7), it will still push at the end with holes in between
console.log(arr2); // [ 5, 6, <8 empty items>, 7 ]
```

.pop()

- This method is used to pop an element from the stack. Here's how it works:

```JS
arr1 = [1, 2, 3, 4];
console.log(arr1); // [ 1, 2, 3, 4 ]
arr1.push(5); // push 5
arr1[arr1.length] = 6; // push 6
console.log(arr1); // [ 1, 2, 3, 4, 5, 6 ]
arr1.pop(); // pop one element
console.log(arr1) // [1, 2, 3, 4, 5]

arr1.length = arr1.length - 1; // also pops
console.log(arr1); // [ 1, 2, 3, 4 ]
// works as expected when no holes exist


// here's how it works when holes exist
```

```
arr2 = [5, 6];
console.log(arr2);
arr2.length = 10; // creates holes
arr2.push(7) // element after holes
console.log(arr2); // [ 5, 6, <8 empty items>, 7 ]
arr2.pop(); // pops 7
console.log(arr2); // [ 5, 6, <8 empty items> ]
arr2.pop(); // removes one hole, NOT element
console.log(arr2); // [ 5, 6, <7 empty items> ]
```

- The `.pop()` method is destructive to the array, which. means it modifies the array itself. It returns the popped element:

```
JS
console.log(arr1.pop()); // prints the popped element, and arr1 is
modified
```

.toString() and .join()

- These two methods are used to convert an array into a string.
- Although they are used to do the same thing, there are small differences between them.
- These methods are non destructive and do not modify the original array. They only return the string representation of an array. Here's how they work:

```
JS
let arr1 = ['This', 'is', 'a', 'test'];
console.log(arr1.toString());  // Outputs: "This,is,a,test"
// The join method allows you to specify a separator
console.log(arr1.join());  // Outputs: "This,is,a,test"
console.log(arr1.join(' '));  // Outputs: "This is a test"
console.log(arr1.join('-'));  // Outputs: "This-is-a-test"
// They also handle nested arrays differently
let arr2 = ['This', ['is', 'a'], 'test'];
console.log(arr2.toString());  // Outputs: "This,is,a,test"
console.log(arr2.join());  // Outputs: "This,is,a,test"
console.log(arr2.join(' '));  // Outputs: "This is,a test"
```

- `toString()` effectively flattens the array before converting it to a string, whereas `join()` preserves the original array structure.

.unshift() and .shift()

- Used for inserting(unshift) and deleting(shift) an element from the front of the array
- It is a destructive function, it modifies the array.
- unshift returns the size of the array after unshifting.
- shift returns the removed element

```JS
arr1 = new Array(10);
console.log(arr1); // [ <10 empty items> ]
arr1.push(10);
arr1.unshift(1);
console.log(arr1); // [ 1, <10 empty items>, 10 ]

arr2 = [1, 2, 3, 5];
console.log(arr2.shift()); // 1
console.log(arr2); // [ 2, 3, 5 ]
console.log(arr2.unshift(10)); // 4 (that is the number of
elements after unshift)
console.log(arr2) // [ 10, 2, 3, 5 ]
```

delete

- Delete is *not* a method but an *operator*, just like `typeof`
- It's used to remove the association from an object's value to the actual value in memory. It does not remove the element from memory. Just removes the association
- Since arrays are also objects, you can use `delete` to remove elements from an array and create undefined holes.
- Since it creates undefined holes, it's not recommended to use the `delete` operator to delete stuff from an array. You should use either `splice`, `pop` or `shift`
- Here's how it works:

```JS
arr1 = [1, 2, 3, 4, 5];
delete arr1[1]; // delete element at index 1
console.log(arr1); // [ 1, <1 empty item>, 3, 4, 5 ]
```

.splice()

- This is an array method that allows you to modify the array.
- The general syntax is like this:

```JS
array.splice(startIndex, deleteCount, [items])
```

- The splice method starts at the `startIndex`, and deletes the number of elements give at `deleteCount`, including the `startIndex`. Optionally, it replaces the deleted elements with the `items` given
- It is a destructive function, it modifies the array. It returns the elements it deleted
- For example:

```JS
// removing elements from an array
let arr = [1, 2, 3, 4, 5];
console.log(arr.splice(2, 2));  // Outputs: [3, 4]
console.log(arr);  // Outputs: [1, 2, 5]

// replacing elements:
arr = [1, 2, 3, 4, 5];
console.log(arr.splice(1, 2, 'a', 'b'));  // Outputs: [2, 3]
console.log(arr);  // Outputs: [1, 'a', 'b', 4, 5]

// adding elements
arr = [1, 2, 3, 4, 5];
console.log(arr.splice(2, 0, 'a', 'b'));  // Outputs: []
console.log(arr);  // Outputs: [1, 2, 'a', 'b', 3, 4, 5]

// if you don't mention deleteCount, it removes all elements till
the end of the array
arr = [1, 2, 3, 4, 5];
console.log(arr.splice(1));  // Outputs: [2, 3, 4, 5]
console.log(arr);  // Outputs: [1]
```

.slice()

- The `.slice` method is used to get a subset from an array. It returns a shallow copy of the sliced array.
- A shallow copy means that it references the same place in memory. I'll tell about the consequences of this in the example
- The general syntax is like this:

```JS
array.slice(start, end)
```

- `start` (Optional value): The index from which the array slicing starts. Element at `start` is included in the spliced array
- If not specified, it defaults to 0.

- `end` (Optional value): The ending index of the array up to which the `slice()` method will select elements.
- The element at the `end` index is not included in the sliced array.
- If not specified, it defaults to the length of the array.
- Here's an example:

```JS
let arr = [1, 2, 3, 4, 5];
let newArr = arr.slice(1, 4);
console.log(newArr);  // Outputs: [2, 3, 4]
console.log(arr);  // Outputs: [1, 2, 3, 4, 5]
```

Consequences of it being a shallow copy

- If you modify the elements of either `arr` or `newArr`, it *will* modify it in both palaces.
- This won't be the case if the array elements are primitive datatypes, like `number` in this case. Because when it's spliced, another copy of those numbers are created. *So modifying `arr` will not modify `newArr` if you're trying to modify a primitive datatype.* This happens because all primitive datatypes are immutable. They can't be changed after they are declared. When you try to change them, they get reassigned under the hood.
- However, it will modify it if you're modifying *an object* inside the array, an object like another array:

```JS
let arr1 = [1, 2, [3, 4], 5, 6]
let newArr = arr1.slice(1); // newArr will be [ 2, [3, 4], 5, 6]

arr1[1] = 10; // modifying primitive datatype, won't reflect in
newArr
arr1[2][0] = 15; // modifying array object inside the array, will
reflect in newArr
console.log(arr1); // [ 1, 10, [ 15, 4 ], 5, 6 ]
console.log(newArr); // [ 2, [ 15, 4 ], 5, 6 ]
```

.concat()

- This method is used to concatenate two arrays
- It is not a destructive function. It does not modify the original array
- It creates a new array and returns it. It is a shallow copy like in the `.slice` method. So all the shallow copy behaviour carries over to the newly created array.
- Here's how it works:

```JS
let arr1 = [1, 2];
let arr2 = [3, 4];
let arr3 = [5, 6];

let new_arr = arr1.concat(arr2);
console.log(arr1); // [ 1, 2 ]
console.log(new_arr) // [ 1, 2, 3, 4 ]

// you can also concat multiple arrays
let another_new_arr = arr1.concat(arr2, arr3);
console.log(another_new_arr); // [ 1, 2, 3, 4, 5, 6 ]
```

.sort()

- This method is used to sort an array.
- It is a destructive function. That means it modifies the original array.
- The general syntax is so:

```JS
array.sort([compareFunction])
```

- The compare function is optional. The default order is ascending.
- It is kinda like the `lambda` function we passed into `map` in Python. But it's used to compare.
- You can use this compare function to sort an array of objects by a specific property of theirs too.
- Here's an example for sorting numbers in an ascending order:

```JS
const numbers = [3, 23, 12];
numbers.sort(function(a, b){ return a - b });
console.log(numbers); // Outputs: [3, 12, 23]
```

- Observe how I gave a compare function even if I wanted them in ascending order
- This is because, if you don't give a compare function, it treats all elements of the array as a string. Which might lead to situations like this:

```JS
let numbers = [33, 16, 156, 2, 9, 5, 10];
numbers.sort();
console.log(numbers); // Output: [10, 156, 16, 2, 33, 5, 9]
```

- Here's an example for comparing a specific property of an object:

```JS
let arr1 = [
  {
    name: 'anurag',
    age: 18
  },
  {
    name: 'cupcake',
    age: 5
  },
]
// array of 2 objects

arr1.sort(function(a, b) {
  return a.age - b.age;
});

console.log(arr1);
// outputs
// [ { name: 'cupcake', age: 5 }, { name: 'anurag', age: 18 } ]
```

.reverse()

- This method is used to reverse a string
- It's a destructive function and modifies the original array
- Example:

```JS
const arr = ['apple', 'banana', 'orange'];
arr.reverse();
console.log(arr); // Outputs: ['orange', 'banana', 'apple']
```

- If you don't want to modify the original array, call `.slice()` on the original array, and then reverse it:

```JS
const arr = ['apple', 'banana', 'orange'];
const reversedArr = arr.slice().reverse();
console.log(reversedArr); // Outputs: ['orange', 'banana',
'apple']
console.log(arr); // Outputs: ['apple', 'banana', 'orange']
```

Functions

- There are, again, no points for guessing, multiple ways of declaring functions too.
- The most straight forward one is the `function` keyword.
- This is how that works:

```JS
function this_is_a_func_name(arg1, arg2){
 // statements
 return some_val;
}
```

- The `function` keyword is kinda like the `def` keyword in Python.
- *Functions are also objects* in JS. As a side effect, you can assign functions to variables and call them:

```JS
const hello = function (name){
 console.log("Hello! " + name);
}

hello("cupcake"); // Hello! cupcake
```

- Notice how I didn't give a name after the `function` keyword. The name is not necessary if you don't need it. This is called an anonymous function. In this case it happens to be assigned to the object/variable `hello`
- Consider the below code:

```JS
const hello = function(name, age) {
  console.log("Hello ", name + " You are " + age + " years old");
}

hello("cupcake");
hello("cupcake", 5);
```

- You'd expect that the first function call would cause an error because `age` is not specified. But JS says nope, we keep going.
- This would be the output of it:

```
Hello  cupcake You are undefined years old
Hello  cupcake You are 5 years old
```

- Arguments which are not passed in are just treated as `undefined`. *They do not cause errors*
- There's also this thing called the `arguments` object inside a function. This is an *array like object* defined by default inside a function. It contains an *array like object* of all the arguments passed to it. This *isn't an actual* array so it doesn't support all the methods of an array. It only pretends to be an *array*, it's an imposter.
- For example:

```JS
function sum() {
  let total = 0;

  for(let i = 0; i < arguments.length; i++) {
    total += arguments[i];
  }
  return total;
}
console.log(sum(1, 2, 3, 4));  // 10
```

- This is useful in coding up variable number of arguments.
- However, this is a bit vague and not properly defined. This can lead to unexpected results and confusion in a bigger program, especially since the `arguments` object isn't a real array
- To get around this, JS provides a more cleaner syntax called a `rest array` defined by `...` (yes, just 3 dots. You'll get it with an example) :

```JS
function greet_and_sum(name,  ... nums) {
  console.log("Hello " + name); // Hello cupcake
  console.log(nums); // [ 1,2,3,4 ]
  console.log("Your sum is: ");
  let total = 0;
  for (let num of nums) {
    total += num;
  }
  return total; // 10
}
console.log(greet_and_sum("cupcake", 1, 2, 3, 4));

// full output
// Hello cupcake
// [ 1, 2, 3, 4 ]
// Your sum is: 10
```

- This is very similar to variable number of arguments in Python. The `...` before a variable name specifies that it's a variable argument type.
- Note that this `rest parameter` must be the last formal parameter, else you will get an error. For example, this isn't allowed:

```JS
function greet_and_sum(name,  ... nums, age) {
 // statements
}
```

Hoisting

- Hoisting is this thing JS does to try to be helpful and shit but it only causes more confusion.
- It's this behaviour of JS that moves variable declarations (*declarations*) only to the top of the scope. This scope can be a function, the main `<script>` tag or whatever your scope may be.
- Here's an example

```JS
console.log(myVar); // Outputs: undefined
var myVar = 5;
console.log(myVar); // Outputs: 5
```

- In the first line, `myVar` is undefined because only the declaration (`var myVar`) was hoisted to the top, not the initialisation (`= 5`). The actual code looks like this to the JavaScript engine:

```JS
var myVar;
console.log(myVar); // Outputs: undefined
myVar = 5;
console.log(myVar); // Outputs: 5
```

- To make things more complicated, the hoisting behaviour is different for `var`, `let`, and `const`.
- `var` is hoisted and initialised with a value of `undefined`. However, `let` and `const` are hoisted but *not initialised.*
- If you try to use a `let` or `const` variable before declaration, you'll get a *ReferenceError*.
- For example:

```JS
console.log(myVar); // ReferenceError!
let myVar = 5;
console.log(myVar); // doesn't run because error encountered at the top
```

- Note that function declarations are also hoisted:

```JS
hello();  // Outputs: 'Hello, world!'
function hello() {
  console.log('Hello, world!');
}
```

- But be careful - function expressions (including those involving arrow functions) are not hoisted:

```JS
hello();  // TypeError: hello is not a function
var hello = function() {
  console.log('Hello, world!');
}
```

- Here, `hello` is hoisted, but the realisation that `hello` is a function comes only after it is assigned to `function() { // stuff }`
- As a good practice, don't rely on hoisting. It's confusing and leads to unexpected errors. Declare your stuff at the top of the scope and then use them appropriately.

Built-in Objects

- There are a couple of objects built into the environment in a browser. These include:

1. `Math`
2. `Number`
3. `String`
4. `Array`
5. `Date`
6. `window`
7. `document`

- The last two are only available on a browser, and not if you're running JS on a server using Node.js
- As far as we are concerned rn, we are only running JS on a browser.

- The `window` object refers to the browser window
- The `document` object refers to the current HTML document being rendered
- A little more info about the `Number`, `String`, `Date`, `Math` and `window` object is given in the slides.

`Number`

The properties of the `Number` object are as follows:

```JS
const biggestNum = Number.MAX_VALUE; // biggest possible value that can hold in memory
const smallestNum = Number.MIN_VALUE; // smallest possible value that can hold in memory (you're limited by the number of digits after decimal places)
const infiniteNum = Number.POSITIVE_INFINITY; // positive infinity
const negativeInfiniteNum = Number.NEGATIVE_INFINITY; // negative infinity
const notANum = Number.NaN; // NaN = Not a Number

// all of the above are just values. There are a few functions too:
const num = 12;
num.toString(); // converts a number to a string, just like an array
```

`String`

- There are a couple of functions and their descriptions given in the form of a table in the slides.
- In real life, you usually loop up documentation whenever you want to use methods like these.
- Here are the methods:

Method	What it does
`.charAt(index)`	returns character at the index given
`.charCodeAt(index)`	same thing as above but returns the unicode value of the character
`.concat(string)`	same as the concat function of arrays but with strings
`fromCharCode(val1, val2,`	expects a variable number of arguments of unicode values. It converts them to a string

Method	What it does
`val3)`	
`.indexOf(substring, index)`	returns the first occurrence of the *substring* starting at the position *index*. if *index* is not given, it searches from index 0
`.lastIndexOf(substring, index)`	same thing as above but in reverse, so it returns the last occurrence from the position *index*. If *index* is not given, it searches from the end of the string
`.slice(start, end)`	works the same as arrays but with strings
`.split(delimeter)`	splits the string into an array of strings separated by the *delimeter*. Similar to `strtok()` in C
`.substr(start, length)`	returns a string containing *length* number of characters from *start* index. If *length* is not specified, it starts at *start* index and returns until the end of the string
`.toLowerCase()`	converts to lowercase
`.toUpperCase()`	converts the string to uppercase
`toString()`	returns the same as the string. It's pointless. Why would you convert a string to a string. Oh right, JS, that's why
`valueOf()`	returns the same as the string. JS provides you two functions to do the same dumb thing. It gives you all the utilities to be stupid

`Date()`

- The date object is used to store dates. You can make a `Date` variable with the `new` syntax:

```JS
let some_date = new Date();
```

- The time in `Date` is stored as Epoch.
- Developers wanted a way to keep track of time. So they did the thing they are most good at. They made a counter.
- This counter counted the number of milliseconds elapsed since Jan 1 1970 00:00:00
- Why this date and time? it was randomly chosen. We developers are bad with decisions like this.
- Then we stored this millisecond count in an integer and called it a day. This is called Epoch time and literally every computer on earth keeps track of time this way. Your TV to your smartphone.
- Google the `Y2K catastrophe` if you have time. It's a really fun read.

- There are a couple of methods of the `Date` object as well.
- In the table below, they are either set or get. What I mean by this is that `(get/set)Date` are two functions, `getDate` and `setDate`. I just combined them into one because I'm lazy to type.
- So you use `getDate` to get the date and `setDate` to set the date
- Ok now the table:

Method	What they do
`.toLocaleString()`	Returns a string of the date information
`.(get/set)Date()`	get or set the day of the *month* from 1 to 31 (unintuitive)
`.(get/set)Month()`	get or set the month, from 0 to 11
`.(get/set)Day()`	get or set the day of the week, from 0 to 6
`.(get/set)FullYear()`	get or set the year
`.(get/set)Time()`	get or set the Epoch number of milliseconds (the absolute epoch value, the counter we talked about earlier)
`.(get/set)Hours()`	get or set the number of hours, from 0 to 23
`.(get/set)Minutes()`	get or set the minutes, from 0 to 59
`.(get/set)Seconds()`	get or set the seconds, from 0 to 59
`.(get/set)Milliseconds()`	get or set milliseconds, from 0 to 999

`Math()`

- The `Math()` object provides a few constants and functions:

Method/ Constant	What it is
`Math.E`	Euler's constant. e^x wala e
`Math.PI`	Pi ka value
`Math.SQRT2`	Square root of 2
`Math.abs(x)`	Returns the absolute value of x
`Math.ceil(x)`	Returns the smallest integer greater than x. In simple words, the nearest integer above the given value
`Math.floor(x)`	Returns the floored value of x
`Math.max(x, y, z)`	Returns the max value in the variable number of args . It will return NaN if the array is nested
`Math.min([array])`	Same thing as above but min value
`Math.pow(x, y)`	returns the value of x^y
`Math.random()`	returns a pseudo random number between 0 and 1

`Window()`

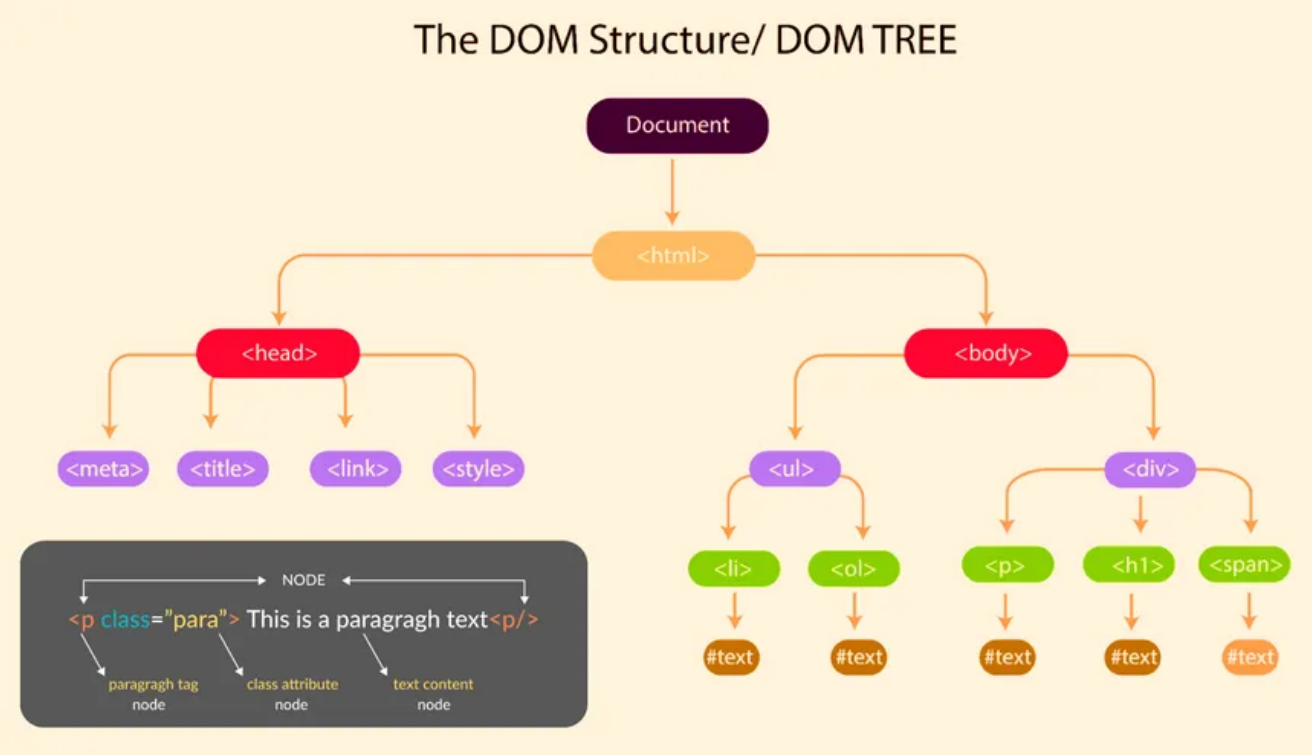
- Global variables declared can also be accessed with `window.x` since it's the main parent object.
- Here are the methods:

Method	What it is
`window.location`	contains information like the current document's URL, hostname, protocol, port etc
`window.history`	contains the browser history (ahem)
`window.localStorage`	allows you to access the browser's local storage cache
`window.innerHeight` and `window.innerWidth`	gives the dimensions of the display area of the browser
`window.alert("some string")` or just `alert("some string")`	Sends an alert box to the user with the passed in string as the message
`window.prompt("question string", default)`	To ask a question to the user. Returns the answer string
`window.confirm("text")`	To show a confirmation dialog

Method	What it is
`window.setInterval()` and `window.clearInterval()`	used to start or stop a performing an action repeatedly after an interval
`window.setTimeout()` and `window.clearTimeout()`	used to start or stop performing an action after a timeout period

DOM Manipulation

- `DOM` isn't *dom*inant, it's short for **D**ocument **O**bject **M**odel. How sad.
- This is a representation of how your HTML elements are there in your HTML document.
- This is represented as the root of a tree. (people just call this a tree)



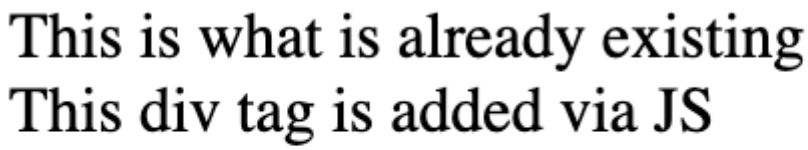
- Every html tag we used will be a node. You can nest HTML tags inside one another, forming a tree.
- You can access these elements in JavaScript and manipulate them. This is how JS is allowed to act as a brain and control the HTML document. This is called *DOM Manipulation*.
- There are various methods/functions to do this:

`document.write`

- This method allows you to append HTML to the end of your HTML document that you've written manually.
- Here's an example:

JS

```
<!DOCTYPE html>
<html lang="en">

<head>
  <title></title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1">
  <link href="css/style.css" rel="stylesheet">

</head>

<body>
  <div>This is what is already existing</div>
  <script>
    additionalHtml = "<div>This div tag is added via JS</div>"
    document.write(additionalHtml);
  </script>
</body>

</html>
```

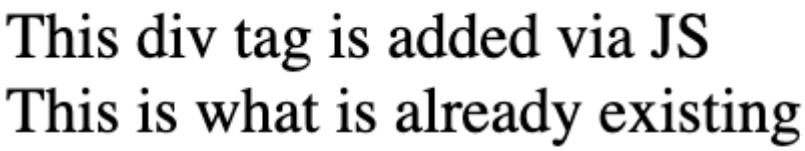


- Be careful that the placement of the `script` tag matters. If I put this `script` tag in the `<head>` section, it will run before any HTML is loaded.
- As a result, it'll be the first HTML tag instead of appending to the last:

HTML

```
<!DOCTYPE html>
<html lang="en">

<head>
  <title></title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1">
```

```
  <link href="css/style.css" rel="stylesheet">
  <script>
    additionalHtml = "<div>This div tag is added via JS</div>"
    document.write(additionalHtml);
  </script>
</head>

<body>
  <div>This is what is already existing</div>
</body>

</html>
```



Selectors

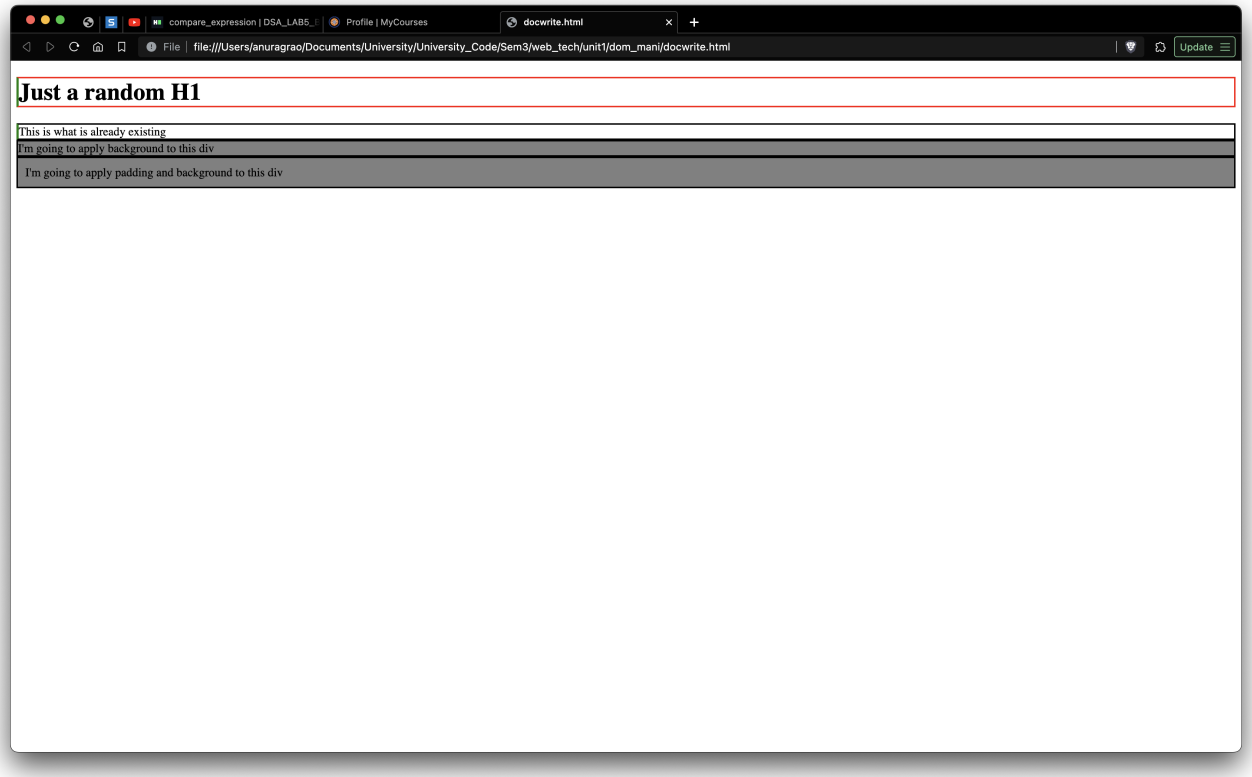
- These are used to select specific nodes. They return a reference to the HTML node(s).
- Then you can manipulate them. In this example, I am modifying their CSS so that the changes are visible

JS

```
<!DOCTYPE html>
<html lang="en">

<head>
  <title></title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1">
</head>

<body>

  <h1 class="apply_color_color_border">Just a random H1</h1>
  <div class="apply_color_color_border">This is what is already
existing</div>
  <div class="apply_background">I'm going to apply background to
```

```
this div</div>
  <div class="apply_background" id="padding">I'm going to apply
padding and background to this div</div>


// script tag must be after html declarations in the body tag so
that this runs only after the html is rendered!

  <script>
    const div_padding = document.getElementById("padding"); //
getElementById (notice it's singular. IDs are meant to be unique)
    div_padding.style.padding = "10px";


    const divs_background =
document.getElementsByClassName("apply_background"); //
getElementsByClassName (notice it's plural. "Elements". There may
be multiple elements with the same class name)
    // loop for every div
    for (const div of divs_background) {
      div.style.backgroundColor = "grey"
    }


    const all_divs = document.getElementsByTagName("div");
    for (const div of all_divs) {
      div.style.border = "2px solid black"; // applying border to
all divs
    }


    // document.querySelector is all of these functions combined.
You can use any CSS selector on it:

    const color_color_border =
document.querySelector(".apply_color_color_border"); // singular.
Selects the first match for the selector
    color_color_border.style.border = "2px solid red"


    const color_color_borders =
document.querySelectorAll(".apply_color_color_border"); // selects
all elements with that selector
```

```
    for (element of color_color_borders) {
      element.style.borderLeft = "3px solid green" // this not
only sets the second div's border left to green but also overrides
the h1's border left only
    }


  </script>
</body>

</html>
```



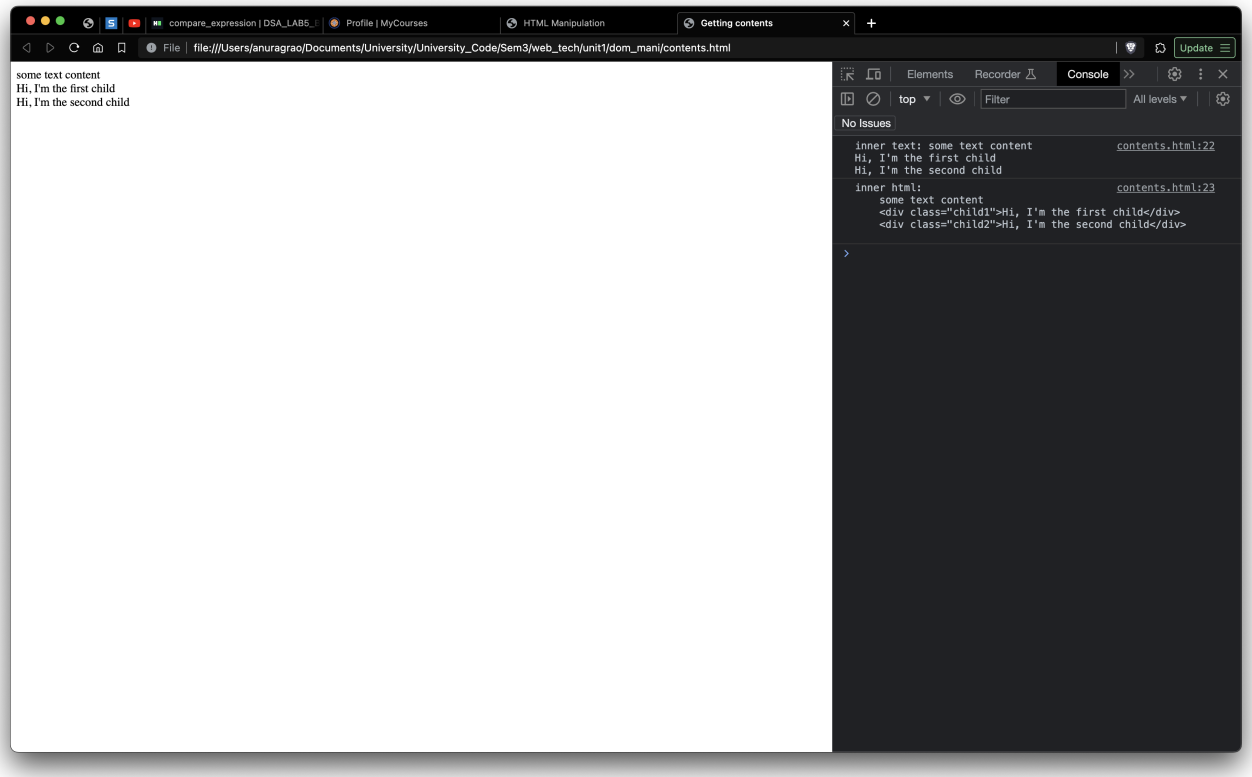
Getting Content

- You can get the contents of a node using these methods. There are only two:

JS

```
<!DOCTYPE html>
<html lang="en">

<head>
  <title>Getting contents</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1">
</head>
```

```
<body>
  <div class="parent">
    some text content
    <div class="child1">Hi, I'm the first child</div>
    <div class="child2">Hi, I'm the second child</div>
  </div>


  <script>
    const parent = document.querySelector(".parent");


    // logging both innerText and innerHTML to the console
    console.log("inner text: " + parent.innerText);
    console.log("inner html: " + parent.innerHTML);
  </script>
</body>

</html>
```



- Observe the console on the right hand side:

```
inner text: some text content                    contents.html:22
Hi, I'm the first child
Hi, I'm the second child
inner html:                                      contents.html:23
    some text content
    <div class="child1">Hi, I'm the first child</div>
    <div class="child2">Hi, I'm the second child</div>

>
```

Modifying HTML

- There are various methods/functions to modify HTML.
- Here's the base code. I will be showing code snippets one by one after this that would show the functions one by one:

JS

```
<!DOCTYPE html>
<html lang="en">

<head>
  <title>HTML Manipulation</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1">
</head>

<body>

  <div class="parent">
    <div class="child1">I'm the first child</div>
    <div class="child2">I'm the second child</div>
  </div>


  <script>
    // creating nodes:

    //creating html node:
    const child3 = document.createElement("div");
    child3.innerText = "I am child 3. I was just created (ahem)";
```

```
    const child4 = document.createElement("div");
    child4.innerText = "I am child 4. I'm gonna replace child2.";
    // creating text node:
    const text_child = document.createTextNode("Hi! I am the text
node. I am just text, not a node. Idk why they call me text
node");



    const parent = document.querySelector(".parent");
    const child2 = document.querySelector(".parent > .child2");
    // .parent > child2 is a selector. I am trying to select
.child2 who would be the child of .parent. I could have just given
.child2 and it would still work since there is only one child2 in
the entire doc. To target things better, we use the > syntax to
select .child2 which is inside .parent only

  </script>
</body>

</html>
```

- The functions I'll mention now are gonna be in the inserted inside the script tag:

JS
```
// appendChild is used to append a child as the last child to a
parent node
parent.appendChild(text_child);
```

I'm the first child
I'm the second child
Hi! I am the text node. I am just text, not a node. Idk why they call me text node

JS
```
// insertBefore is used to insert a node before a specific child
node
// syntax: insertBefore(newNode, referenceNode)
parent.insertBefore(child3, child2); // child 3 goes in between
child 1 and child 2. Don't ask me why
```

I'm the first child
I am child 3. I was just created (ahem)
I'm the second child
Hi! I am the text node. I am just text, not a node. Idk why they call me text node

```JS
// replacing child2 with child4
// syntax: replaceChild(newChild, oldChild)
parent.replaceChild(child4, child2);
```

I'm the first child
I am child 3. I was just created (ahem)
I am child 4. I'm gonna replace child2.
Hi! I am the text node. I am just text, not a node. Idk why they call me text node

```JS
// removing a child. let's remove child 4. too much cock it's showing
// syntax: removeChild(child)
parent.removeChild(child4);
```

I'm the first child
I am child 3. I was just created (ahem)
Hi! I am the text node. I am just text, not a node. Idk why they call me text node

```JS
// lastly, let's remove the parent only
parent.remove();
// it removes the parent node itself. Along with parent, all it's children are also gone
```

Here's the entire code for reference:

JS

```
<!DOCTYPE html>
<html lang="en">

<head>
  <title>HTML Manipulation</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1">
</head>

<body>

  <div class="parent">
    <div class="child1">I'm the first child</div>
    <div class="child2">I'm the second child</div>
  </div>


  <script>
    // creating nodes:

    //creating html node:
    const child3 = document.createElement("div");
    child3.innerText = "I am child 3. I was just created (ahem)";
    const child4 = document.createElement("div");
    child4.innerText = "I am child 4. I'm gonna replace child2.";
    // creating text node:
    const text_child = document.createTextNode("Hi! I am the text
node. I am just text, not a node. Idk why they call me text
node");


    const parent = document.querySelector(".parent");
    const child2 = document.querySelector(".parent > .child2");
    // .parent > child2 is a selector. I am trying to select
.child2 who would be the child of .parent. I could have just given
.child2 and it would still work since there is only one child2 in
the entire doc. To target things better, we use the > syntax to
select .child2 which is inside .parent only


    // appendChild is used to append a child as the last child to
```

```
a parent node
    parent.appendChild(text_child);

    // insertBefore is used to insert a node before a specific
child node
    // syntax: insertBefore(newNode, referenceNode)
    parent.insertBefore(child3, child2); // child 3 goes in
between child 1 and child 2. Don't ask me why

    // replacing child2 with child4
    // syntax: replaceChild(newChild, oldChild)
    parent.replaceChild(child4, child2);

    // removing a child. let's remove child 4. too much cock it's
showing
    // syntax: removeChild(child)
    parent.removeChild(child4);

    // lastly, let's remove the parent only
    parent.remove();


  </script>
</body>

</html>
```

Events

- Events are basically triggers. They are mostly user interactions.
- For example, clicking a button, hovering over something, double click on an element are events.
- We usually make a function that fires every time an event is triggered. This function is called an *event handler*
- The process of connecting events to an event handler is called *registration*
- There are 2 ways to implement this thing:

1. Assign event handlers to html elements, aka *inline event handlers*
2. Use *event listeners*

Inline Event Handlers

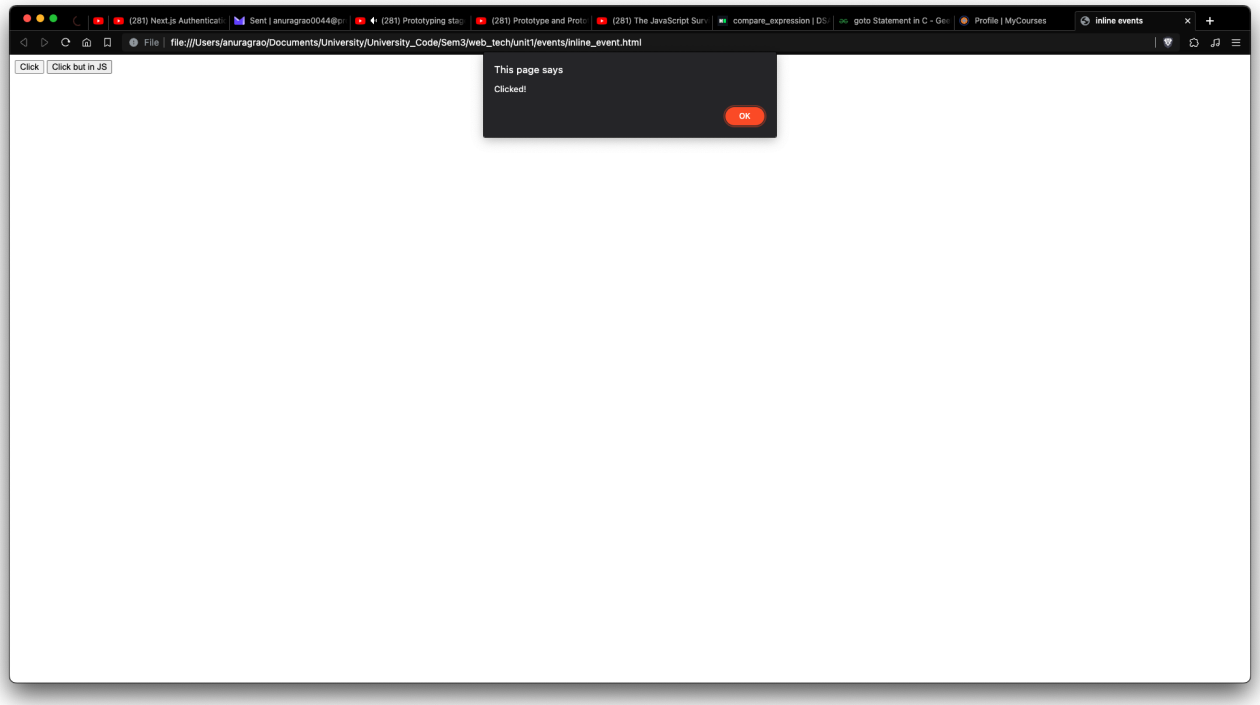
- In this method, we add function with the `on<event>` syntax. This could be `onclick`, `ondblclick`, `onkeypress` etc.
- You can either do this in HTML or in JS.

HTML

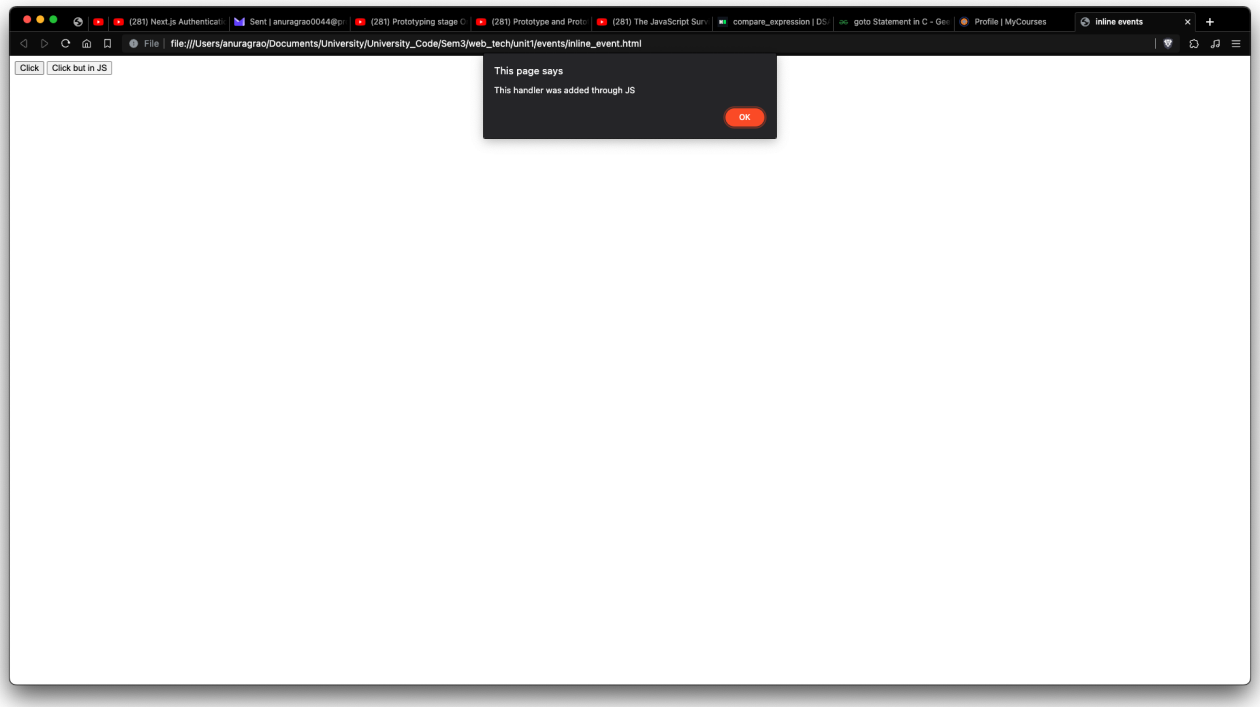
```
<!DOCTYPE html>
<html lang="en">

<head>
  <title>inline events</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1">
</head>

<body>
  <button type="button" onclick=handleClick()>Click</button>
  <button type="button" id="js-button">Click but in JS</button>


  <script>
    function handleClick() {
      alert("Clicked!");
    }

    // does the same thing but separated these two functions for
explanation purposes
    function handleClicked_JS() {
      alert("This handler was added through JS");
    }

    const butt = document.querySelector("#js-button")
    butt.onclick = handleClicked_JS; // notice the absense of ().
We are setting the onclick property of the butt object to be
handleClicked_JS (object, function is an object). Putting () calls
the function and appoints the return value to the butt.onclick
property
  </script>
</body>

</html>
```

Click Click but in JS

on clicking the `Click` button:



on clicking the `Click but in JS` button:



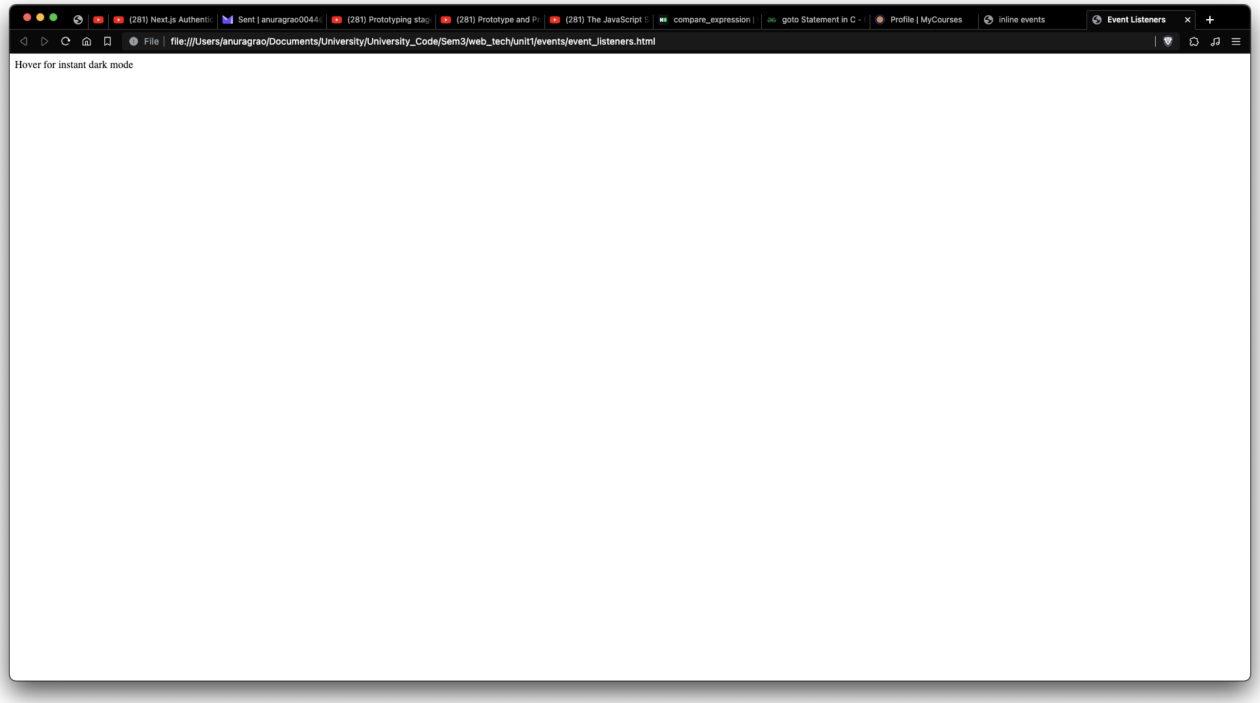
Event Listeners

- An event listener *listens* for an event on a particular element.
- The syntax is like this:

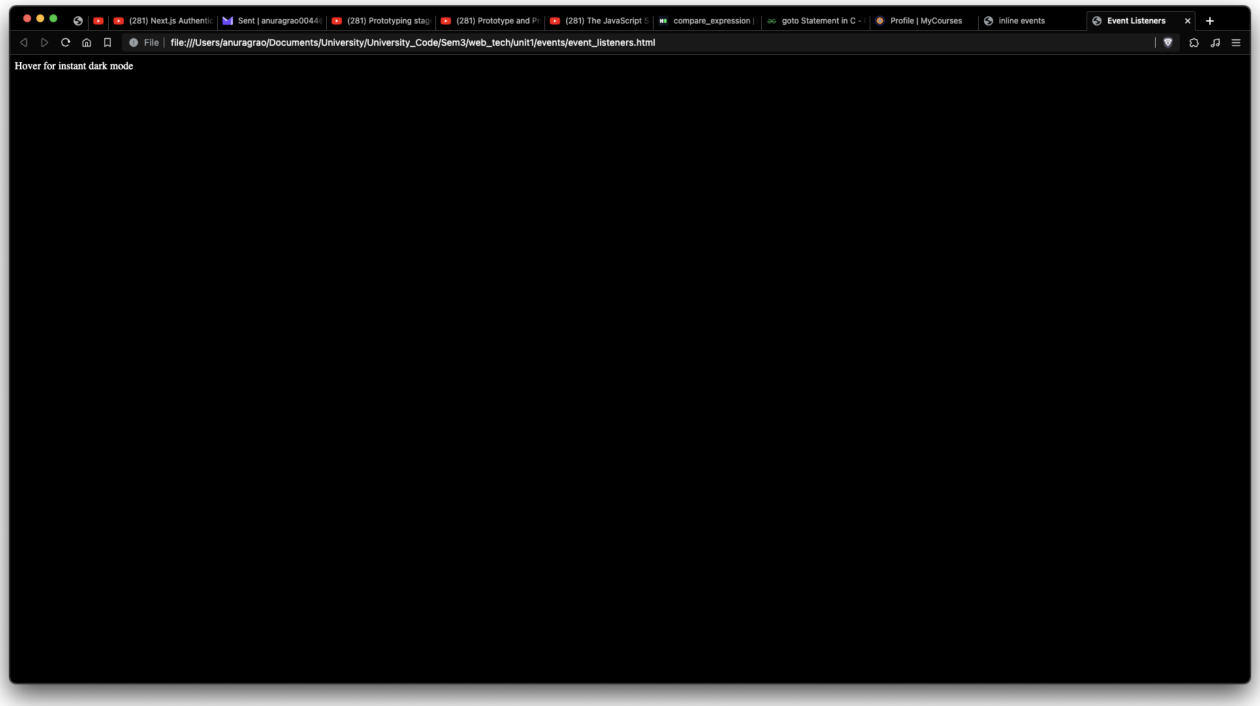
```JS
element.addEventListener(event, handler);
// event is the event it should listen for. Can be "click",
"mouseover", etc.
```

For example:

```HTML
<!DOCTYPE html>
<html lang="en">

<head>
  <title>Event Listeners</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1">
</head>

<body>
  <div id="some_id">
    Hover for instant dark mode
  </div>

  <script>
    const div = document.querySelector("#some_id");
    function handleMouseOver() {
      const body = document.querySelector("body");
      body.style.backgroundColor = "black";
      div.style.color = "white"; // so that you're still able to
see the text on a black background
    }

    div.addEventListener("mouseover", handleMouseOver);
  </script>
</body>

</html>
```



On hovering over the text:



Here's a bunch of events for reference:

Source	Event	Fires When...
Mouse	click	the mouse is clicked and released on an element
	dblclick	an element is clicked twice
	mousemove	every time a mouse pointer moves inside an element
	mouseover	every time a mouse pointer is placed over an element
Keyboard	keydown	when a key is pressed down
	keyup	when a key pressed is released
	keypress	when a key is pressed and released
Form	submit	a form is submitted
	reset	a form reset button is clicked
	focus	an input element is clicked and receives focus
	blur	an input element loses focus

The `event` object

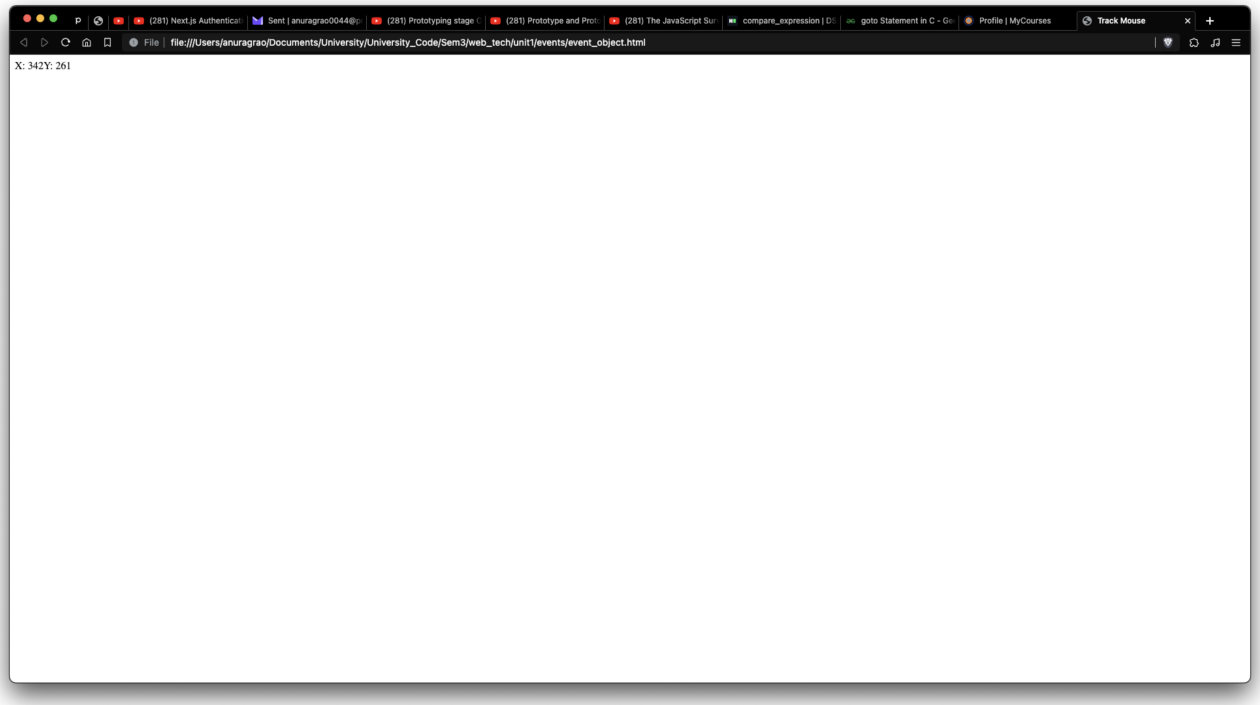
- event listener functions receive an `event` object by default.
- You can use this `event` object to get information about the event that happened.
- Here's an example of how you can use this:

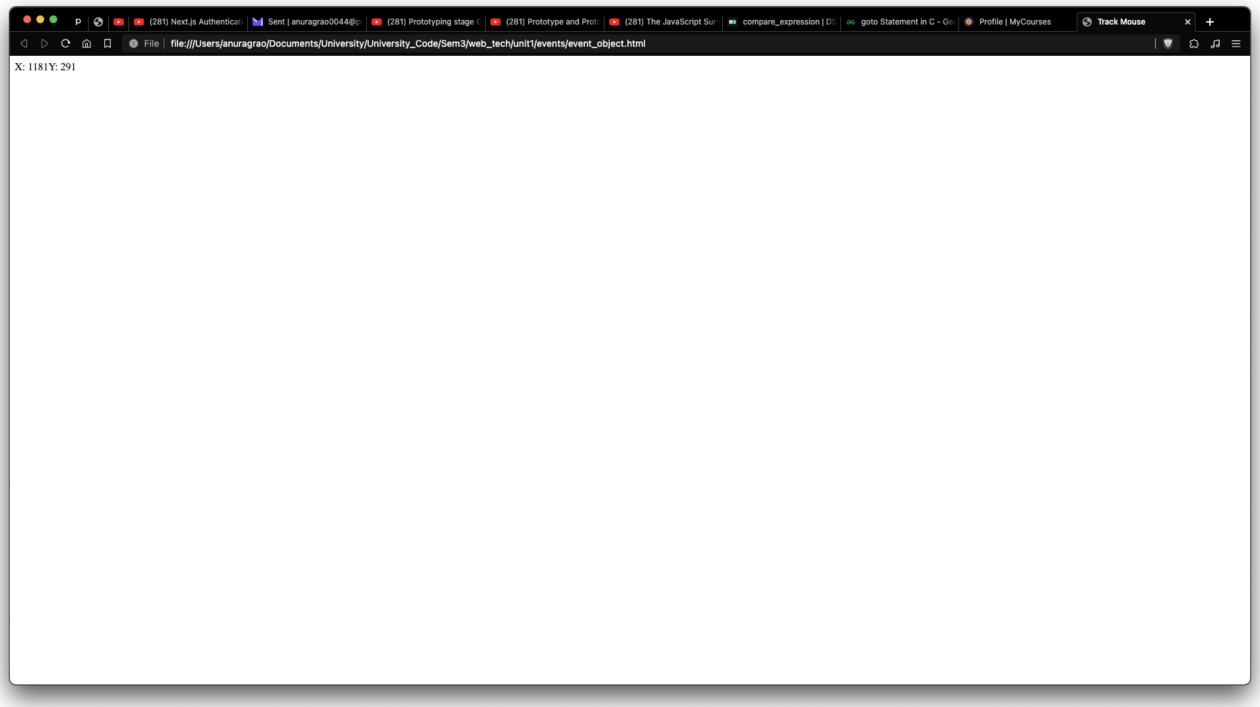
HTML

```
<!DOCTYPE html>
<html lang="en">

<head>
  <title>Track Mouse</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1">
</head>

<body>
  <div class="xy">X: 0 Y: 0</div> <!-- this will show the current
x and y positions of the pointer -->
  <script>
    function handleMouseOver(event) {
      const divxy = document.querySelector(".xy");
      let x = event.clientX; // get x and y positions
      let y = event.clientY;
      divxy.innerText = "X: " + x + "Y: " + y; // set it to the
inner text of the div
    }
```

```
    document.onmousemove = handleMouseOver; // on mouse move on
the entire document, fire handleMouseOver
  </script>
</body>

</html>
```

You can check out a live demo of this at [codepen]





- My pointer isn't visible in the screenshots but every time I move my pointer, the function triggers.
- A really cool practical use case of this is to have some kind of object that follows your mouse.
- Anyways, here's the bunch of properties you can access using the `event` object:

Property	IE5-8 Equivalent	Specifies
target	srcElement	the target of the event (most specific element).
type	-	the name of event fired (without the on prefix)
altKey / shiftKey / ctrlKey / metaKey	-	true/false to signify if Alt Key or Shift Key or Ctrl Key or Meta Key was pressed
charCode	keyCode	Unicode character code of the pressed key
key	-	Key Character Name (‘a’ or ‘F1’ or ‘CAPS LOCK’)
button	-	Returns which mouse button was pressed
clientX, clientY / offsetX, offsetY / screenX, screenY	-	the coordinates of the mouse pointer when the event triggered, relative to, the current window / target element / screen

Event Propagation

- When you have elements nested inside other elements, which often is the case in real world applications, events become a little more complicated.
- When you trigger an event on an element, what happens to it's parents?
- While developing, I faces this issue of event propagation. I had a card, and a button inside those cards. When the button was clicked, the event listener on the card also triggered.
- This is called event propagation, where the event is propagated between parent and child elements

There are 3 ways, or *phases* which an event can propagate to handlers:

1. Capturing phase:

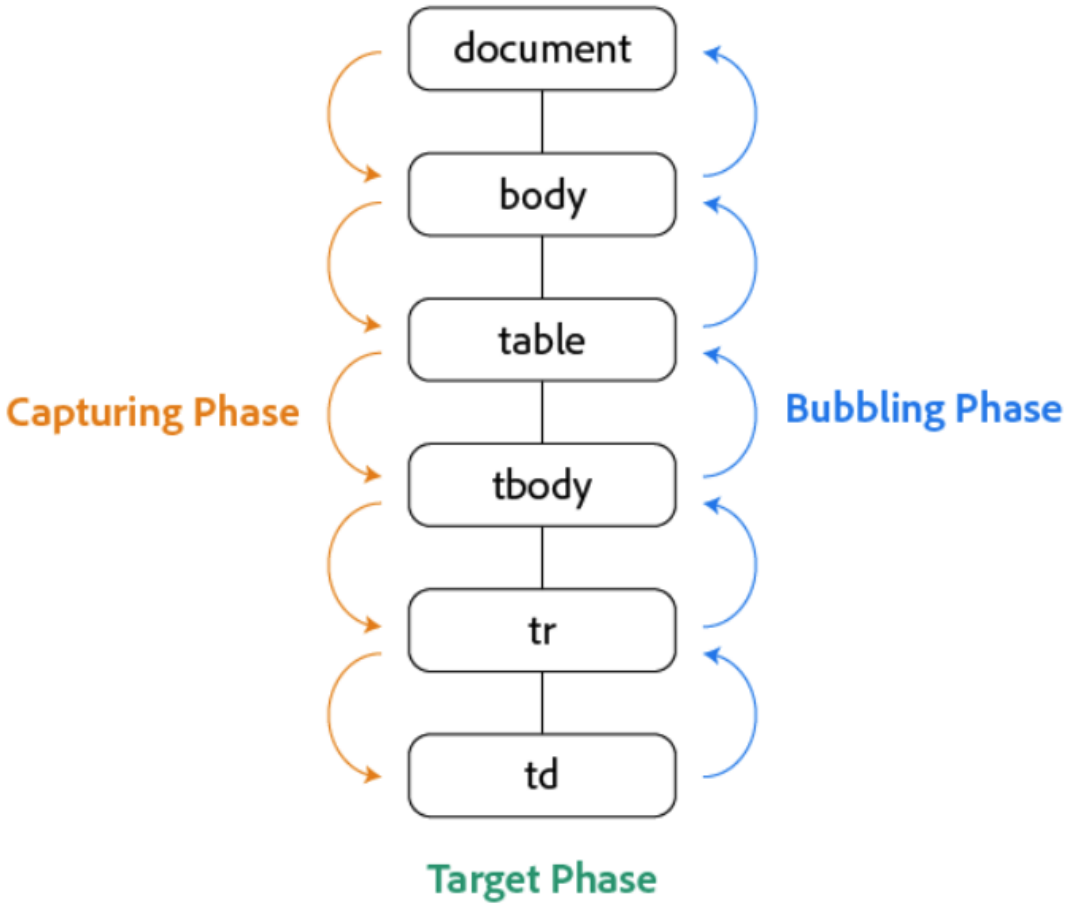
- The events trigger from the parent elements to the child elements. The event travels from the outermost parent to the child. All the event listeners are triggered on the way to the target element.

2. Target phase

- Also known as 'at' phase, the event listener attached directly to the target element is triggered here. No propagation happens

3. Bubbling phase

- This is the exact reverse of capturing phase. The events get triggered in the child element first, then it's parent, then it's parent, until we get to the outermost parent.
- You'd often see this when clicking on table elements:



- When you click on td, In the capturing mode (phase), the event listeners attached to `document` get triggered first, then `body`, then `table`, and so on.
- If you use the target mode (phase), the event listener attached to `td` only gets triggered
- If you use the bubbling mode (phase), the event lister attached to `td` gets triggered first, then `tr`, then `tbody`, and so on. It *bubbles* up to the parent elements.
- By default, events are in the bubbling phase. You can change this in the `addEventListener()` function like this:

```
JS
element.addEventListener("event", handlerFunction, flag);

// The last argument, 'flag', defines if the bubbling phase is
used or not. By defualt, this if false. If you set this to true,
it will use the capturing phase/capturing mode.
// for example:
```

```
// imagine this element is nested inside other elements
const ele = document.querySelector(".some_classname");
function handleClick(){
 alert("clicked");
}

ele.addEventListener("click", handleClick, true); // will use
capturing phase
```

- You can also use the `event` listener object to access some functions of event propagation:

Property/ Method	IE5-8 Equivalent	Purpose
cancelBubble	-	A historical alias to stopPropagation(). Setting its value to **true** before returning prevents propagation
eventPhase	-	Specifies which phase of the event flow is being processed
cancelable	Not supported	Indicates whether you can cancel the default behaviour of an element
preventDefault()	returnValue	It cancels the default behavior of the event (if possible)
stopPropogation()	cancelBubble	It stops any further bubbling/ capturing of the event.

- If you remember the web tech assignment we did, we had to trigger an `alert()` function whenever the submit button was clicked.
- The submit button's default behaviour is to clear the form, send the request and open and reload the page. We didn't want that. We needed to only do the `alert` and not reload the page.
- So we used the `event.preventDefault()` method inside the event handler for the submit button to prevent it's default behaviour.

Made with ♡ by Anurag Rao